



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE LAS CIENCIAS DE COMPUTACIÓN

IMPLEMENTACIÓN DE UN SERVIDOR DE NOMBRES USANDO LA LIBRERÍA
DNS-RUST

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

NOMBRE COMPLETO AUTOR

PROFESOR GUÍA:
JAVIER BUSTOS

MIEMBROS DE LA COMISIÓN:
NOMBRE COMPLETO UNO
NOMBRE COMPLETO DOS
NOMBRE COMPLETO TRES

SANTIAGO DE CHILE
2025

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Una dedicatoria corta.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Tabla de Contenido

1. Introducción	1
1.1. Contexto	1
1.2. Glosario	2
1.3. Objetivos	3
2. Marco teórico	5
2.1. RFC's	5
2.1.1. RFC 1034	5
2.1.2. RFC 1035	5
2.1.3. RFC 4033	6
2.1.4. RFC 4034	6
2.2. Estado del Arte	6
2.2.1. ISC BIND server heap overflow de 4 bytes	6
2.2.2. Vulnerabilidades en Dnsmasq	7
2.2.3. Por qué seguir los RFC	8
2.2.4. Otras implementaciones de Name Servers en Rust	8
3. Conexiones UDP y TCP	10
3.1. UDP	10
3.2. TCP	14
4. Estructura y procesamiento de datos	15

4.1.	Masterfile	15
4.2.	Catalogo	16
4.3.	Hash anidado de Resource Records Set (RRset)	17
4.4.	Procesamiento del Masterfile	18
4.4.1.	read_masterfile_records	18
4.4.2.	parse_resource_records	20
4.4.3.	group_rrs_into_rrsets	22
4.5.	Descarte del árbol jerárquico	23
4.5.1.	RFC 1035	23
4.5.2.	Problemas de implementación	23
5.	Algoritmo de Búsqueda	24
5.1.	Flujo y funcionamiento	24
5.1.1.	find_zone_in_catalog	26
5.1.2.	resolve_qname_in_zone	26
5.1.3.	add_glue_records_and_wildcard	29
5.1.4.	add_rrsigs_for_delegation	30
5.1.5.	add_rrsigs_for_answer	31
5.1.6.	add_nsecs_proofs	32
5.2.	Resultados según cada caso de consulta	34
5.2.1.	Consulta con coincidencia exacta con rtype y el nombre de dominio .	34
5.2.2.	Consulta con coincidencia solo en el nombre de dominio, pero existe un registro CNAME	34
5.2.3.	Consulta que sale de la zona autoritativa : Referral con Glue Records	35
5.2.4.	Consulta sin coincidencia en el nombre de dominio pero existe un wildcard	36
5.2.5.	Consulta sin coincidencia en nombre de dominio y sin wildcard: NX-DOMAIN	36
5.2.6.	Consulta sin coincidencia en el tipo y sin wildcard: NODATA	37

6. EDNS0	38
6.1. Características principales de EDNS0	38
6.2. Ejemplo de mensaje DNS con EDNS0	39
7. DNSSEC	40
7.1. Servidor	40
7.1.1. Generación de llaves criptográficas	40
7.1.2. Estructuras de datos para las llaves	42
7.1.3. Inserción de DNSKEY's RRSETS en la estructura de datos	44
7.1.4. Inserción de los RRSIG de los DNSKEY's RRSET	45
7.1.5. Creación e inserción de firmas y el manejo de casos bordes	46
7.1.6. Creación e inserción de los registros NSEC	47
7.1.7. Inserción de RRSIGS para los NSEC RRSET	48
7.1.8. Creación del RRSET DS para la zona padre	48
7.2. Resolver	49
7.2.1. Proceso de autenticación para nombres inexistentes (NXDOMAIN) .	51
7.2.2. Proceso de autenticación para respuestas NO_DATA	52
7.2.3. Autenticación para respuestas provenientes de Wildcards	53
7.2.4. Proceso de Autenticación DNSSEC Estándar	55
7.2.5. Proceso de verificación de la cadena de autenticación	57
7.2.6. Ejemplo real de validación DNSSEC	60
8. Discusiones	64
Bibliografía	66
Apéndice A. Anexo	67

Índice de Tablas

Índice de Ilustraciones

Capítulo 1

Introducción

1.1. Contexto

DNS (Domain Name System) es un protocolo fundamental para el funcionamiento de la capa siete del modelo OSI, permitiendo que un usuario pueda acceder a una IP global a través de un nombre de dominio, como por ejemplo dcc.uchile.cl. Para lograr esto existen 2 actores principales: el Resolver y el Name Server.

El Resolver es el programa encargado de tomar una consulta de un nombre de dominio desde un dispositivo como un computador o un celular y transformarlo en una consulta DNS, la cual es un mensaje en bytes con una estructura definida por el protocolo DNS, con las siguientes 5 secciones: Header, Question, Answer, Authority y Additional

El Name Server es el que conoce las IP de los dominios y es el encargado de responder las consultas DNS que envía el Resolver, pero este proceso no es sencillo y requiere del arduo estudio y programación de los RFC (Request For Comments), documentos estandarizados de cómo debe funcionar el internet.

En este contexto, el área de investigación de NIC Chile (NIC Labs) implementó una librería de funciones para DNS en lenguaje Rust, como caso de prueba de dicha librería desean implementar un servidor de nombres (Name Server) DNS que utilicen para su propia zona niclabs.cl. Una zona es un conjunto de nombres de dominios hijos de un dominio padre, en este caso el dominio padre es niclabs.cl (la zona misma) y un dominio hijo podría ser a.niclabs.cl

La librería DNS-RUST de NIC Labs sigue cabalmente las indicaciones de los RFC sin implementar ningún tipo de optimización que no sea referenciada por ellos, por lo que es importante la implementación del Name Server bajo esas especificaciones ya que puede ser utilizado más adelante para investigación y desarrollo.

Mencionar que la librería DNS-RUST es solo una herramienta que contiene las funciones necesarias para transformar un mensaje DNS en bytes a una estructura en Rust (parse), un Cliente y un Stub-Resolver, es decir, no posee nada relacionado a un Name Server. De esta

manera todas las funcionalidades, estructuras de datos y algoritmos serán implementados desde cero.

Como se verá más adelante, implementar un Name Server con todas sus funcionalidades incluyendo las que aún no posee la librería es un trabajo que toma más de un semestre, de hecho, para tener más contexto, el Stub-Resolver de la librería DNS-RUST fue desarrollado por años de manera grupal, por lo que es lógico pensar que su contraparte (un Name Server) no puede ser completado al 100 por ciento por una sola persona en un semestre, provocando así que el trabajo a realizar sea acotado sólo a las funcionalidades de DNS Security Extension, Extension DNS 0, creación de estructuras de datos basadas en el `masterfile`, implementación del algoritmo de búsqueda del Name Server y conexiones UDP-TCP asíncronas.

1.2. Glosario

Antes de seguir con el desarrollo de este informe, es necesario abarcar ciertos conceptos específicos que se referenciarán más adelante:

- Stub-Resolver: Es un resolver que utiliza un resolver recursivo, el cual se encarga de todo el proceso real para responder a una consulta DNS. Estos resolvers recursivos son complejos y un stub-resolver suele utilizar los creados por Google y otras grandes empresas.
- Extension DNS 0 (EDNS0): Primera extensión del protocolo DNS, incluyendo la capacidad de aumentar el tamaño de los mensajes DNS (consultas y responses).
- DNS Security Extension (DNSSEC): Extensión de seguridad del protocolo DNS, permitiendo mensajes firmados y validaciones basadas en criptografía.
- Resource Record (RR): Estructura de datos que se envía como respuesta desde el Name Server. Existen muchos tipos, cada uno con sus especificaciones.
- RRset: Conjunto de RR's que poseen el mismo nombre de dominio, tipo y clase.
- Catalogo: Estructura jerárquica con las zonas que maneja el Name Server.
- Arbol de dominios: Estructura jerárquica que contiene los RRSets de los dominios guardados por el Name Server.
- Tipos de Resource Records clásicos:
 - Answer (A): Dirección IP del nombre de dominio solicitado.
 - Name Server (NS): Nombre de dominio del Name Server autoritativo que sabe responder a la consulta solicitada.
 - Canonical Name (CNAME): Nombre de dominio secundario-canónico que pueda tener el nombre de dominio solicitado.

- Optional (OPT): RR especial para Extension DNS 0 con muchas funcionalidades, pero la más importante es la de aumentar el tamaño default de las consultas DNS (512 bytes).
- Tipos de Resource Records de DNSSEC:
 - DNS Public Key Record (DNSKEY): Contiene la llave pública necesaria para validar la firma contenida en el RRSIG.
 - Resource Record Signature (RRSIG): Firma criptográfica creada a partir de una llave privada alojada en el Name Server.
 - Delegation Signer (DS): Hash de la llave pública alojada en el Name Server padre necesario para validar el DNSKEY RR.
 - Next Secure Record (NSEC): RR que contiene el siguiente nombre de dominio en un orden canonico dentro del arbol de dominios (ordenado en bytes).
- Clase en un RR: Valor que determina el medio a usar en el protocolo, normalmente es de tipo *IN* (Internet), pero también existen otros tipos secundarios que no es necesario mencionar.
- RDATA (Response Data): Valores de retorno que contienen la información clave de un RR. Es diferente para cada tipo de RR.
- Glue Record: RR especial que contiene la IP de un Name Server delegado en otra zona, usado para evitar ciclos de referencias.
- Wildcard: Nombre de dominio especial que permite que subdominios no definidos respondan con la misma información, usando un asterisco (*). Por ejemplo, *.niclabs.cl puede responder a consultas como a.niclabs.cl o b.c.niclabs.cl.
- NXDOMAIN : Código de error que indica que el nombre de dominio consultado no existe.
- NO_DATA : Código de error que indica que el nombre de dominio consultado existe, pero no posee registros del tipo solicitado.

1.3. Objetivos

Objetivo General

El objetivo de este trabajo es construir un MVP (Producto Mínimo Viable) de un Name Server Autoritativo Iterativo capaz de controlar la zona de dominio del laboratorio de manera segura, eficiente, siguiendo al pie de la letra los RFC y usando las herramientas de la librería DNS-RUST como apoyo.

Que el Name Server sea Iterativo consiste en que su algoritmo busca el nombre de dominio solicitado dentro de su estructura de datos, si no tiene la respuesta directa, envía una

referencia hacia otro Name Server que si podría tenerla, y si no encuentra ninguna respuesta envía un error. La alternativa es un Name Server recursivo que, si no conoce la respuesta a la consulta, utiliza un Resolver interno para averiguarla y responder, y que por su diseño y construcción no es parte de este trabajo de título.

Objetivos Específicos

1. Establecer canales de transmisión confiables, en los protocolos de UDP y TCP asincrónicos.
2. Responder correctamente a consultas DNS de tipo basico (A, NS, CN) con sus Resource Records correspondientes.
3. Seguir estrictamente lo establecido por los RFC antiguos y actuales.
4. Asegurar seguridad con DNSSEC (DNS Security Extension), usando algoritmos de encriptación, llaves publicas y privadas, firmas y una cadena de autenticación.
5. Crear de estructuras de datos jerárquicas (árboles) a partir de un `masterfile` .
6. Conectar las herramientas de la librería DNS-RUST usando su stub-resolver como cliente y sus parses de mensajes para manejar las consultas DNS .
7. Cumplir con estándares de documentación en el código.
8. Incluir EDNS0 (Extension DNS 0) necesaria para cumplir DNSSEC. Especificamente sabiendo leer y procesar consultas del tipo OPT.

Capítulo 2

Marco teórico

2.1. RFC's

Parte fundamental del protocolo DNS son los RFC, ya que definen las reglas y el comportamiento que debe seguir todo el internet en general, y en específico el protocolo DNS. A continuación se detallan los RFC más importantes para el desarrollo de este proyecto:

2.1.1. RFC 1034

- Estructura de una consulta DNS, en específico, el formato de la sección Header.
- Resource Records, explica campos como el owner name, tipo, clase, TTL (Time To Live) y RDATA (Response Data)
- Algoritmo de búsqueda de un Name Server para un nombre de dominio.
- Algoritmo de consulta de un Resolver.
- WildCards, nombres de dominios especiales que permiten que subdominios no definidos respondan con la misma información.

2.1.2. RFC 1035

- Significado de los valores y flags dentro del Header.
- Formatos de los RDATA de tipos comunes de RR (A, SOA, NS, CNAME).
- Conexiones UDP y TCP y sus usos en cada caso.
- `masterfiles` y como a partir de estos crear las estructuras de datos.
- Arquitectura del Name Server, específicamente el uso de la concurrencia-asincronismo en el procesamiento de consultas.

2.1.3. RFC 4033

- Glosario de conceptos DNSKEY: Cadena de autenticación, pares de llaves criptográficas, puntos de delegación, etc.
- Dos nuevos bits en el Header: Checking Disabled (CD) y Authenticated Data (AD).
- Uso de EDNS0 para aumentar el tamaño base (512 bytes) de una consulta DNS. Esto ya que las llaves y firmas usadas en DNSSEC suelen ser de gran tamaño.

2.1.4. RFC 4034

- Formatos de los RDATA de los tipos de DNSSEC (DNSKEY, RRSIG, DS y NSEC)
- Forma Canonica para ordenar los RRSets y los nombres de dominios.
- Algoritmos de encriptación
- Algoritmo para calcular el Key Tag del RRset de DNSKEY

2.2. Estado del Arte

En el contexto actual existen distintos software para crear un Name Server, algunos comerciales como Bind9 [?] que está escrito en C y que es Open Source y otros como PowerDNS [?] que es de código privado. Estas son herramientas útiles, pero no es posible verificar si cumplen estrictamente con las especificaciones de los RFC, lo que resulta especialmente importante para NIC Labs. Además, estos programas no están escritos en Rust, lenguaje de programación muy valorado por su alto nivel de seguridad en accesos a memoria [?].

Es importante tener seguridad en el acceso a memoria, ya que previamente se han encontrado o aprovechado vulnerabilidades de Name Servers escritos en otros lenguajes. A continuación se verán algunos ejemplos:

2.2.1. ISC BIND server heap overflow de 4 bytes

Como se vio previamente Bind9 es uno de los software más usados para crear Name Servers, pero a pesar de esto en febrero de 2021 se reportó y se parchó una vulnerabilidad que se mantuvo por 15 años sin ser explotada, y que fue descubierta por un investigador anónimo [?].

La vulnerabilidad permite de manera remota y sin autenticación envenenar el caché del server (Poisoned Cache Attack), provocando así que el caché guarde y responda IPs a sitios inseguros a consultas legítimas, afectando gravemente a todos los usuarios que consulten a este Name Server.

La vulnerabilidad nace por el siguiente código escrito en C:

```

1 static int
2 der_get_oid(const unsigned char *p, size_t len, oid *data, size_t *size) {
3 // ...
4 data->components = malloc(len * sizeof(*data->components)); // <-- (1)
5 if (data->components == NULL) {
6     return (ENOMEM);
7 }
8 data->components[0] = (*p) / 40; // <-- (2)
9 data->components[1] = (*p) % 40;
10 --len; // <-- (3)
11 ++p;
12 for (n = 2; len > 0U; ++n) {
13     unsigned u = 0;
14
15     do {
16         --len;
17         u = u * 128 + (*p++ % 128);
18     } while (len > 0U && p[-1] & 0x80);
19     data->components[n] = u; // <-- (4)
20 }
21 // ...
22 }

```

La función provoca un overflow de 4 bytes al final del arreglo, permitiendo al atacante utilizar esos 4 bytes para inyectar código malicioso y provocar el envenenamiento del caché.

Con el lenguaje Rust habría sido más difícil que ocurriera, ya que el compilador hubiese prevenido al programador al momento de asignar un valor fuera de los límites del vector original.

2.2.2. Vulnerabilidades en Dnsmasq

Dnsmasq [?] es un redireccionador de DNS usado por servidores que en primera instancia no quieren usar la red global de Name Servers y utilizar únicamente el caché proporcionado por este servicio. En general se usa para sistemas embebidos con zonas muy pequeñas que no necesitan mucha comunicación con otros servidores.

El problema con este software escrito en el lenguaje C es que presentó múltiples vulnerabilidades simultáneamente, en concreto se encontraron 4 vulnerabilidades de buffer overflow (provocadas por errores directos en el código) y 3 vulnerabilidades de validación que podrían provocar envenenamiento del caché, a continuación se muestran las últimas 3 vulnerabilidades:

- CVE-2020-25681: No se validaba el par (dirección, puerto) ni el ID de la consulta DNS al recibirlas.
- CVE-2020-25685: Usaba un algoritmo de hashing débil (CRC32), vulnerable a ataques.
- CVE-2020-25686: Dnsmasq no chequeaba que una consulta hubiese terminado de procesarse para recibir y redireccionar otra igual (con el mismo nombre de dominio, tipo y

clase), por lo que el atacante puede enviar muchas consultas equivalentes tratando de adivinar el ID para que así el cliente la acepte como válida, y esto no es tan improbable por la paradoja del cumpleaños [?]

A pesar de que estas 3 vulnerabilidades mencionadas no sean consecuencia directa del lenguaje usado, sino de la mala verificación y uso de algoritmos, es fundamental conocerlas y analizarlas para evitar repetir errores ya cometidos.

2.2.3. Por qué seguir los RFC

Los RFC son los documentos estandarizados que explican cómo debe ser implementado el internet en sus diferentes capas, pero, ¿Qué consecuencias puede tener ignorar las reglas de estos documentos?

A continuación se verán vulnerabilidades descubiertas en Name Servers que no siguieron indicaciones **obligatorias** dadas por los RFC.

Falta de validaciones en los nombres de dominio

Según el RFC 1035 [?] los nombres de dominio deben seguir un estándar y cumplir ciertas reglas, en específico deben cumplir lo siguiente:

- Tener un largo de máximo 255 bytes en total.
- Cada label dentro del nombre de dominio puede tener un máximo de 63 bytes.
- Terminar con un carácter \0.

Todas esas condiciones deben ser verificadas, de lo contrario pueden ocurrir diversos ataques de buffer overflow, ataques de Denegación de Servicio (DDoS) y ejecución de código remoto [?].

En concreto se descubrió una vulnerabilidad [?] consecuencia de la falta de validación del largo de un nombre de dominio dentro de la implementación de DNS en Nut/OS en un dispositivo Ethernut (dispositivo de IoT). Solamente con esta falla se podría ejecutar código remoto y/o un ataque DDoS.

De manera similar, se encontraron muchas vulnerabilidades en dispositivos IoT causadas por la falta de validación del largo de los labels del nombre de dominio [?].

2.2.4. Otras implementaciones de Name Servers en Rust

Existen otras librerías de DNS (en específico con el módulo de Name Server) que están siendo desarrolladas por otras organizaciones, como HickoryDNS [?] y domain[?] de

NLnetLabs. Estas librerías están en un punto muy alto de desarrollo, pero es importante conocer y comprender cuáles han sido sus errores. A continuación se verán algunos ejemplos:

Hickory vulnerabilidad DNSSEC por verificación de un solo DNSKEY

En el Resolver de Hickory al momento de validar un DNSKEY a través del ancla de confianza (trust anchor), que es un registro ya guardado en el Resolver para terminar la cadena de autenticación, si el DNSKEY es validado, por error se valida cualquier otro DNSKEY proveniente del mismo RRSset [?], lo cual es incorrecto, ya que cada uno de estos debe ser validado de forma individual.

Error al firmar la zona debido a glue records

Para firmar una zona (aplicando DNSSEC) se necesita encontrar el SOA RR, el cual permite verificar que la zona es legítima, y este RR siempre debe estar en el Apex Zone (en la zona más alta de la estructura de datos jerárquica), pero si hay algún glue RR (los cuales tienen las IP de otros Name Servers) estos pueden anteceder al SOA debido al ordenamiento canónico (por bytes).

Este reordenamiento provocaba que el proceso de firma no encontrara el SOA RR y por lo tanto no firmara una zona válida [?].

Capítulo 3

Conexiones UDP y TCP

El servidor DNS implementado usa la librería `tokio` para atender consultas en UDP y TCP de forma asíncrona y concurrente. Además de la funcionalidad básica de recibir y devolver respuestas, también soporta EDNS0, detecta y respeta el payload máximo anunciado por el cliente en el registro OPT, maneja el estándar DNSSEC (lectura del bit DO) y protege operaciones críticas con timeouts para evitar bloqueos y bucles infinitos. También se manejan correctamente errores de parseo (se responde FORMERR) y se implementa el comportamiento para truncamiento (TC) sobre UDP con fallback a TCP.

A continuación se describen las implementaciones específicas para cada protocolo.

3.1. UDP

El servidor UDP está implementado en la estructura `ServerUDPConnection`, que contiene los siguientes campos y métodos:

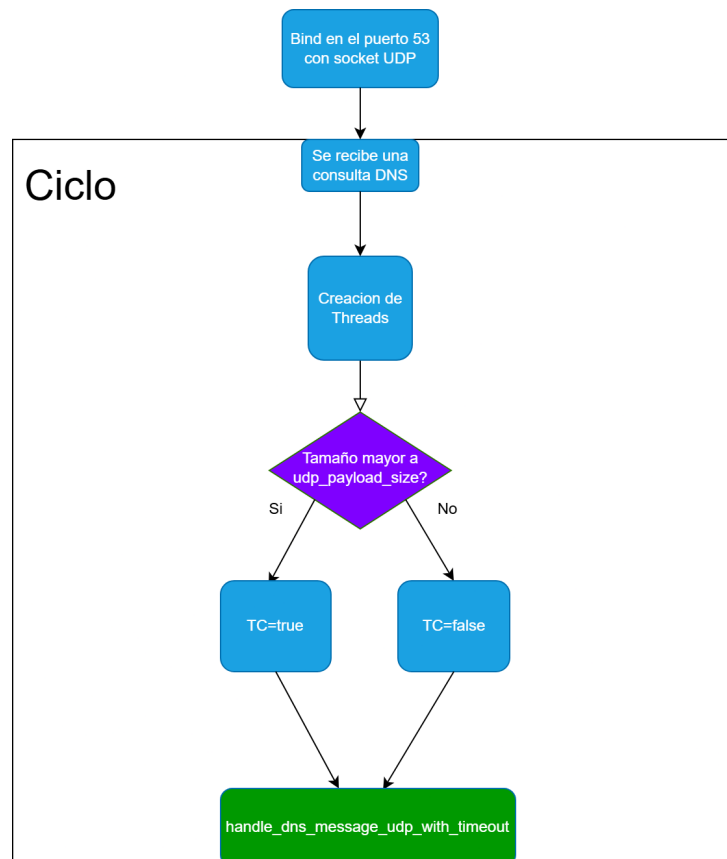
ServerUDPConnection
Campos
- <code>timeout: tokio::time::Duration</code> - <code>payload_size: usize</code>
Métodos Públicos
+ <code>new_udp(timeout: Duration, payload_size: usize) ->Self</code> + <code>new_default(timeout: Duration) ->Self</code> + <code>receive_and_response(&mut self) ->Result<(), ServerError></code>
Métodos Privados / Auxiliares
- <code>handle_dns_message_udp_with_timeout (</code> <code>timeout_server: Duration, recv_buf: &[u8],</code> <code>socket: &UdpSocket, client_addr: SocketAddr,</code> <code>tc_case: bool, len: usize) ->Result<(), ServerError></code>

El campo `timeout` define el tiempo máximo que el servidor esperará para recibir una consulta o enviar una respuesta antes de abortar la operación. El campo `payload_size` establece el tamaño máximo de la consulta DNS que el servidor puede procesar.

Los métodos definidos incluyen funciones típicas como `new` y `new_default`, que permiten crear una instancia de la clase con parámetros personalizados o por defecto. Sin embargo, la función más relevante es `receive_and_response`, la cual se encarga de recibir y responder de manera asíncrona a las consultas DNS enviadas por los clientes. Además esta función tiene en cuenta el tamaño máximo que puede soportar el servidor al momento de recibir consultas UDP, ya que si el tamaño de la consulta excede el `payload_size` se activa el bit TC en la respuesta para que el cliente reintente la consulta por TCP.

La función `receive_and_response` sigue los siguientes pasos:

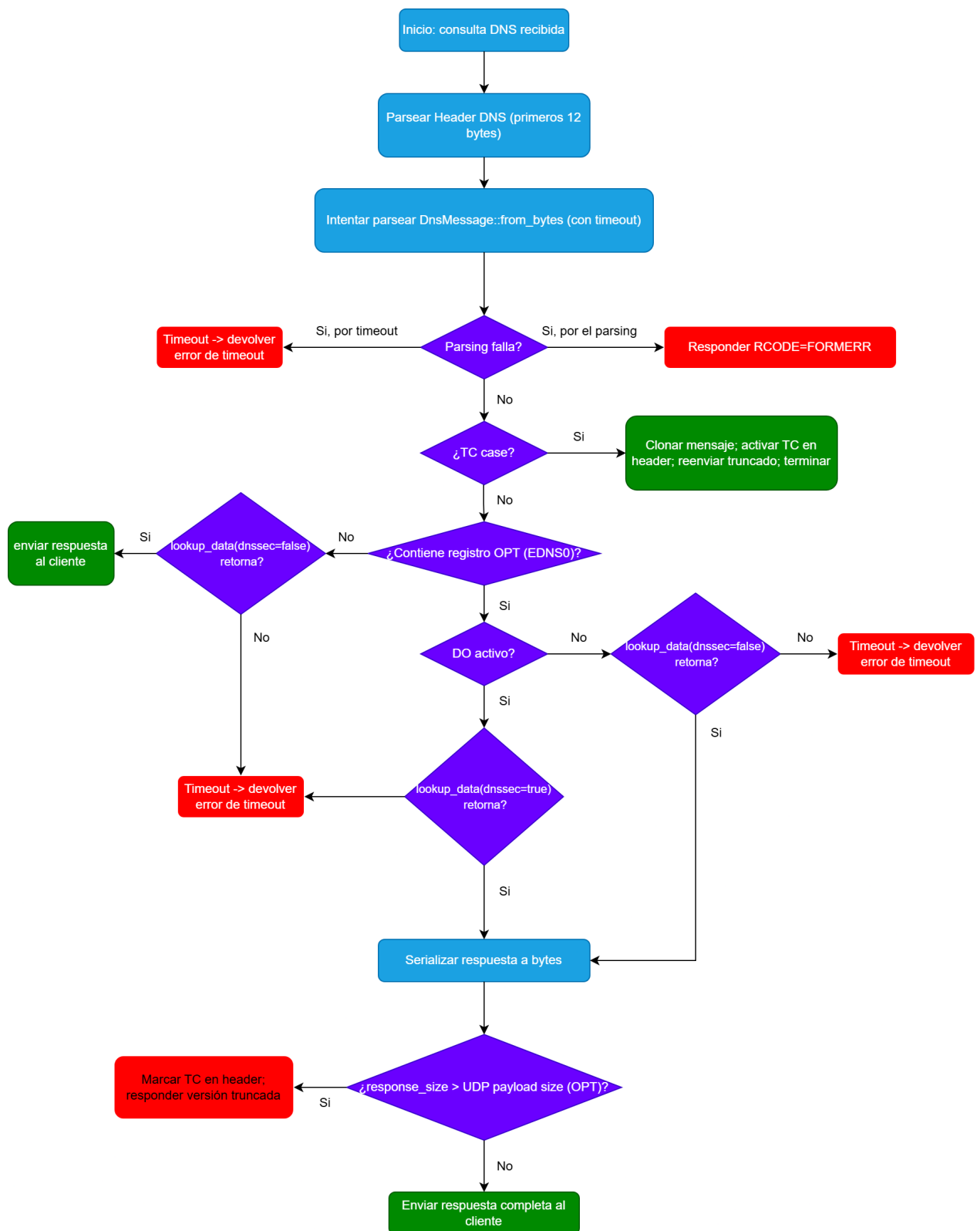
- Crea un socket UDP y se enlaza a localhost en el puerto 53.
- Entra en un bucle infinito donde se espera recibir consultas DNS.
- Crea threads con `tokio` para manejar cada consulta de forma concurrente.
- En cada thread, si el tamaño de la consulta es mayor al `payload_size`, se usa la función `handle_dns_message_udp_with_timeout` con `tc_case` en `true`, de lo contrario se llama a la misma función con `tc_case` en `false`. `tc_case` indica si se debe manejar el caso de truncamiento (TC).



Con respecto a la función auxiliar `handle_dns_message_udp_with_timeout`, esta se encarga de:

- Parsear el **Header DNS**, correspondiente a los primeros 12 bytes del buffer recibido.
- Intenta parsear el **mensaje DNS completo** bajo un `timeout`.
 - Si falla el parsing, responde con un `RCODE=FORMERR`.
 - Si expira el timeout, devuelve un error de timeout.
- Si `tc_case` es verdadero:
 - Clona el mensaje original.
 - Activa el bit **TC** en el header de respuesta.
 - Reenvía el mensaje original truncado al cliente.
 - Termina la ejecución.
- Si no es un TC case, busca si el mensaje contiene un registro **OPT** (EDNS0):
 - Si hay algún **OPT**, obtiene el **DO bit** desde el campo `TTL`.
 - Dependiendo del DO bit:
 - Si está activo, llama a `lookup_data` con `dnssec=true`.
 - Si no está activo, llama a `lookup_data` con `dnssec=false`.
 - Transforma la respuesta en bytes.
 - Compara el tamaño de la respuesta con el valor de `UDP payload size` del registro **OPT**.
 - Si la respuesta es mayor al tamaño permitido por el cliente:
 - Marca el TC bit en la cabecera original.
 - Responde con la versión truncada.
 - Si cabe dentro del tamaño permitido:
 - Envía la respuesta completa.
- Si no hay EDNS0 (sin registro **OPT**):
 - Llama a `lookup_data` con `dnssec=false`.
 - Envía la respuesta al cliente.

El diagrama de flujo de la función `handle_dns_message_udp_with_timeout` se muestra a continuación:



3.2. TCP

El servidor TCP está implementado en la estructura `ServerTCPConnection`, que contiene los siguientes campos y métodos:

ServerTCPConnection
Campos
- <code>timeout: tokio::time::Duration</code> - <code>payload_size: usize</code>
Métodos Públicos
+ <code>new(timeout: Duration, payload_size: usize) ->Self</code> + <code>new_default(timeout: Duration) ->Self</code> + <code>get_timeout(self: Self) ->tokio::time::Duration</code> + <code>receive_and_response(&self) ->Result<(), ServerError></code>
Métodos Privados / Auxiliares
- <code>handle_dns_message_with_timeout(timeout_server: Duration, recv_buf: &[u8], socket: &mut TcpStream) ->Result<(), ServerError></code> - <code>handle_connection(timeout_server: Duration, socket: TcpStream, addr: std::net::SocketAddr) ->Result<(), ServerError></code>

La versión TCP del servidor comparte gran parte de su estructura con el modo UDP, ya que utiliza los mismos campos principales y métodos similares para la configuración y el procesamiento de consultas.

La principal diferencia radica en la función `handle_connection`, la cual se encarga de gestionar cada conexión TCP entrante. Esta función primero lee un prefijo de 2 bytes que indica la longitud total del mensaje DNS (requisito del protocolo cuando se utiliza TCP) y, a continuación, recibe el mensaje completo. Una vez obtenido, delega su procesamiento a la función `handle_dns_message_with_timeout`.

En este caso la función `handle_dns_message_with_timeout` no necesita manejar el caso de truncamiento (TC), ya que TCP garantiza la entrega completa del mensaje. Además es prácticamente idéntica a la vista en UDP pero con la diferencia que antes de mandar el mensaje final (sea el caso que sea) debe anteponer el prefijo de 2 bytes con la longitud del mensaje a enviar.

Capítulo 4

Estructura y procesamiento de datos

4.1. Masterfile

Un masterfile es un archivo de texto que contiene los registros DNS de una zona en un formato estandarizado. Se trata de la representación más básica y tradicional para almacenar la información de una zona. Cada línea del archivo corresponde a un Resource Record, cuyos campos están separados por espacios o tabulaciones. Además, el formato admite directivas especiales que permiten definir parámetros globales para la zona, como el valor predeterminado del TTL (Time To Live) o el servidor de nombres autoritativo.

A continuación se muestra un ejemplo con datos ficticios para la zona example.com:

```
1 example.com.      IN SOA ns1.example.com. hostmaster.example.com. (  
2                    2025052101 ; serial  
3                    3600      ; refresh  
4                    1800      ; retry  
5                    1209600    ; expire  
6                    3600      ; minimum TTL  
7                    )  
8 example.com.      IN NS  ns1.example.com.  
9 example.com.      IN NS  ns2.example.com.  
10 *.example.com.    IN A  0.0.0.0  
11  
12 www.example.com.  IN A    1.2.3.4  
13 www.example.com.  IN A    1.1.1.2 ; second IP  
14 mail.example.com. IN A    2.2.2.2  
15 blog.example.com. IN CNAME blog2.example.com.  
16 blog2.example.com. IN A    1.2.3.4  
17 ns1.example.com.  IN A    3.3.3.3  
18  
19 b.example.com.     IN NS  ns.a.b.example.com. ; delegation point  
20 ns.a.b.example.com. IN A  4.3.2.1 ; name server address (glue)
```

Listing 4.1: Contenido del masterfile de la zona example.com

Como se aprecia en el ejemplo, es posible incluir comentarios utilizando el símbolo `;`; asimismo, cuando un registro es extenso, se permite dividirlo en múltiples líneas encerrando su contenido entre paréntesis `()`, lo que mejora significativamente la legibilidad. Incluso dentro de estos paréntesis se pueden insertar comentarios adicionales.

En resumen, un masterfile almacena la información en texto plano, de manera sencilla y legible para humanos. Sin embargo, este formato no es eficiente para responder consultas DNS directamente, por lo que es necesario procesarlo y transformarlo en estructuras de datos internas más adecuadas. A continuación, se describe cómo se realiza este procesamiento y las estructuras utilizadas para representar la información de la zona en memoria.

4.2. Catalogo

Estructura que contiene un `HashMap` donde cada llave es un string con el nombre de la zona y el valor es un puntero a la estructura con los datos reales. Es básicamente un índice que permite acceder rápidamente a los datos de cada zona, y es especialmente útil cuando el servidor maneja múltiples zonas DNS. También es útil para saber si una consulta pertenece a una zona manejada por el servidor o no, ya que si el dominio consultado no es parte de ninguna zona del catalogo, entonces se puede responder inmediatamente con un `REFUSED`.

El valor de retorno en el `HashMap` está envuelto en un `Arc<RwLock<>>` para permitir el acceso concurrente a los datos de la zona desde múltiples threads.

A continuación se muestra la definición de la estructura `Catalog`:

```
1 pub struct Catalog {  
2     zones: HashMap<String, Arc<RwLock<DataStructure>>>,  
3 }
```

4.3. Hash anidado de Resource Records Set (RRset)

Estructura de datos global accesible por el algoritmo de búsqueda para la obtención de los Resource Record Set. En un principio la estructura de datos iba a ser un árbol n-ario jerárquico, pero al momento de implementarlo hubieron muchos problemas, esto se verá con más detalle en la sección (REDACTED)

Esta estructura contiene el nombre de la zona (por ejemplo 'niclabs.cl') y todos los datos asociados organizados en un `HashMap` anidado. La primera llave corresponde al **nombre de dominio**, y la segunda al **Rrtype** (tipo de registro). Esta organización responde al hecho de que el algoritmo de búsqueda debe obtener un **RRset**, el cual está definido como el conjunto de todos los registros (**RR**) que comparten el mismo nombre de dominio, tipo (**Rrtype**) y clase. En esta implementación se asume que la clase es siempre **IN** (Internet), por lo que no se incluye como parte de las llaves del hashmap.

A continuación se muestra la definición de la estructura `DataStructure`:

```
1 pub struct DataStructure {  
2     name: String,  
3     data: HashMap<DomainName, HashMap<Rrtype, RRset>>,  
4 }
```

Aclarar también que para que la estructura sea accesible de manera global y concurrente, se crea la siguiente variable global:

```
1 pub static DATA_STRUCTURE_NICLABS: Lazy<Arc<RwLock<DataStructure>>> =  
    Lazy::new(|| {  
2     let path = 'src/masterfiles/masterfile.txt';  
3     Arc::new(RwLock::new(data_initialization(path, 'niclabs.cl').  
        unwrap()))  
4 });
```

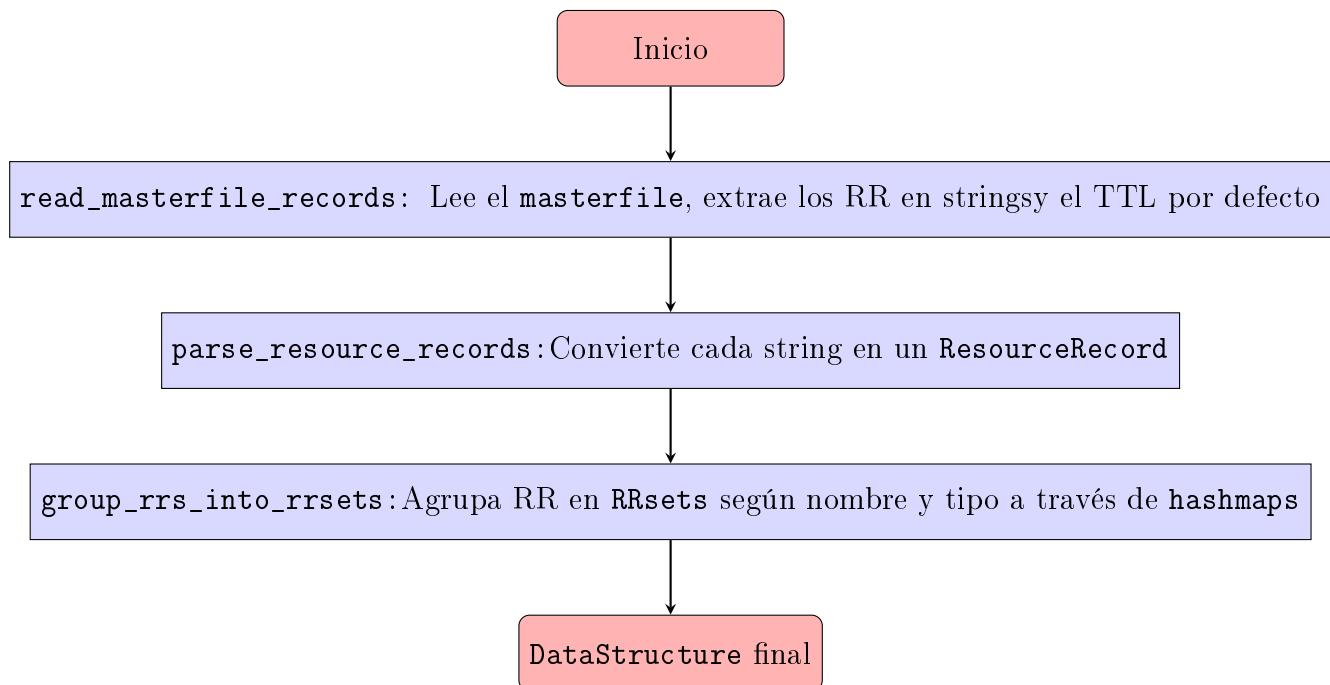
Mencionar que esta variable está envuelta en muchas estructuras, a continuación se explica el uso de cada una:

- **RwLock**: Permite múltiples accesos concurrentes de solo lectura desde distintos hilos (*threads*). Sin embargo, cuando un hilo adquiere acceso de escritura, bloquea todas las lecturas hasta que finalice la operación de escritura.
- **Arc**: Es un puntero inteligente de conteo de referencias atómico que permite compartir datos entre múltiples hilos de manera segura. Es necesario usar un puntero ya que la estructura **Catálogo** debe mantener una referencia compartida a esta información.
- **Lazy**: Es una construcción de evaluación diferida que permite inicializar un valor solo la primera vez que se accede a él. Esto es útil para inicializar estructuras pesadas de forma diferida y segura en tiempo de ejecución.

En primera instancia esa variable global más la misma pero para la zona `example.com` son suficientes, pero como el server emulará zonas padres para lograr la resolución de la cadena de autenticación de DNSSEC, se deberán crear variables globales similares para las zonas `com.`, `cl.` y `root.`.

4.4. Procesamiento del Masterfile

Pasar de la estructura vista en el Masterfile al HashMap anidado de RRset requiere de un proceso extenso de parsing, el cual se puede resumir con el siguiente diagrama:

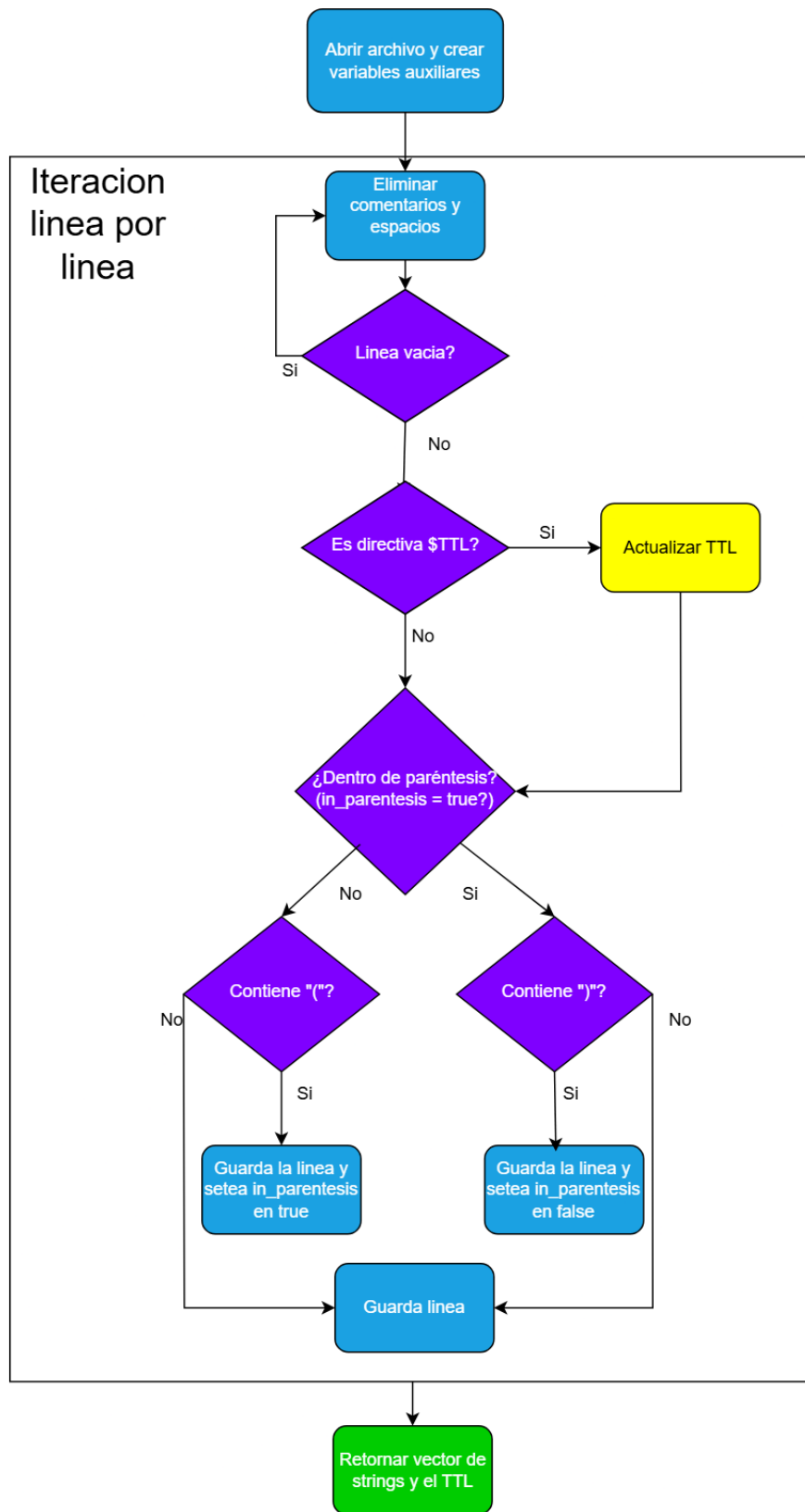


A continuación se describen en detalle cada una de las funciones involucradas en el proceso:

4.4.1. read_masterfile_records

Esta función se encarga de leer el archivo masterfile línea por línea, ignorando comentarios, líneas vacías, manejando los parentesis para registros largos y extrayendo el TTL por defecto si está definido. Cada línea que corresponde a un Resource Record se almacena en un vector de strings, el cual es retornado junto con el TTL por defecto.

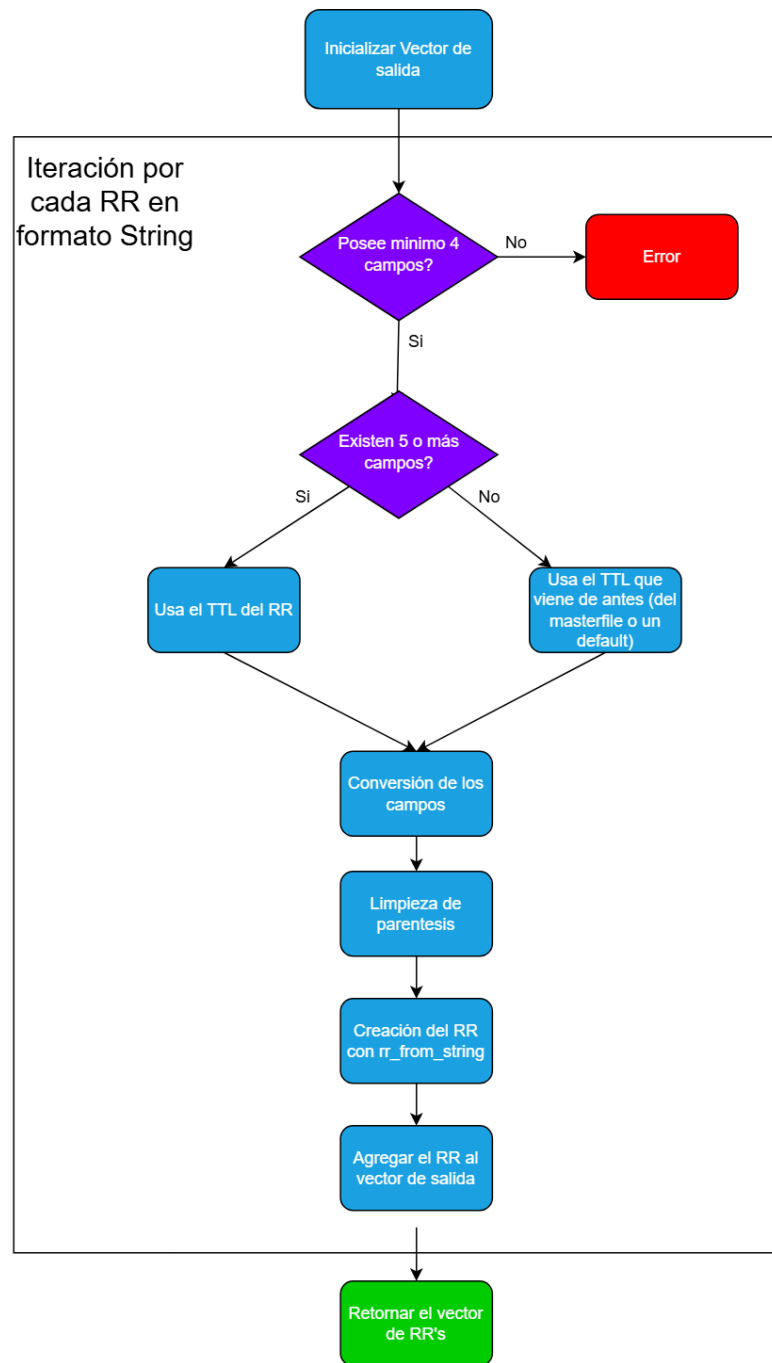
El diagrama de flujo es el siguiente:



4.4.2. parse_resource_records

Esta función recibe el vector de strings generado en la función anterior y convierte cada string en una estructura `ResourceRecord`. Para ello, divide cada línea en sus componentes (nombre, TTL, clase, tipo y datos) y los asigna a los campos correspondientes de la estructura. Si el TTL no está especificado en el registro, utiliza el TTL por defecto extraído previamente.

Su diagrama de flujo es el siguiente:



Como se ve en el diagrama, esta función llama a otra función auxiliar llamada `rr_from_string`, la cual se encarga de convertir un string específico en un `ResourceRecord`. Basicamente esta función hace un match y según el tipo del RR, se llama recursivamente a funciones especializadas para cada tipo de RR. Esto se hizo así ya que en la librería `dns-rust` ya existía ésta función para el caso de tipo A, pero no estaba para los demás, así que se tuvo que implementar para los siguientes tipos:

- A
- AAAA
- CNAME
- MX
- NS
- SOA
- TXT
- DS

En este caso el parametro `rdata_values` corresponde a los datos del Rdata divididos en palabras, es decir, un iterador que permite recorrer cada palabra de los datos del `Rdata`.

Aclarar también que no se muestra cada función especializada para cada tipo de RR por cuestiones de espacio, pero todas siguen la misma lógica: reciben el nombre, TTL, clase y el iterador de datos, y retornan un `ResourceRecord` con los campos correspondientes llenados.

4.4.3. group_rrs_into_rrsets

Esta función recibe el vector de `ResourceRecord` generado previamente y los agrupa en conjuntos `RRset` utilizando una estructura de varios `HashMap` anidados. En primer lugar, la función recorre todos los registros y obtiene, para cada uno, su owner name y su tipo (`Rrtype`). Con ambos valores se construye una llave compuesta que se usa para poblar un primer `HashMap`. Este mapa inicial permite agrupar los registros de acuerdo con la definición de un `RRset` (mismo nombre y mismo tipo), aunque su estructura no es todavía la más conveniente. Por ello, en un segundo paso, se recorre nuevamente este mapa y se genera una nueva estructura jerárquica: el primer nivel corresponde al owner name, el segundo al `Rrtype`, y finalmente, como valor asociado, se almacena un `RRset`. En esta etapa, cada vector de `ResourceRecord` se transforma explícitamente en un objeto `RRset`, ya que, estrictamente hablando, un `RRset` no es simplemente una colección de `RR's`, sino una estructura que encapsula dichos registros junto con metadatos adicionales asociados.

La definición de `RRset` de la librería `dns-rust` es la siguiente:

```
1 pub struct RRset {  
2     name: DomainName,  
3     rrtype: Rrtype,  
4     rclass: Rclass,  
5     ttl: u32,  
6     records: Vec<(u16, Rdata)> //Pair rdlen, rdata  
7 }
```

Como se ve en la estructura, un `RRset` contiene los valores que definen el conjunto (nombre, tipo, clase), un TTL común que suele ser el mínimo de todos los `RR's` y finalmente un vector con tuplas que contienen la longitud del `Rdata` y el `Rdata` en sí. ¿Por qué se guarda el `Rdata` en vez del `ResourceRecord` completo? Porque sería redundante, ya que los valores comunes a todos los `RR's` del conjunto ya están almacenados en los campos superiores del `RRset`.

El diagrama de flujo se encuentra en anexos (REDACTED).

4.5. Descarte del árbol jerárquico

Como se vio en la sección Hash anidado de Resource Records Set (RRset) se intentó implementar un árbol n-ario jerárquico para almacenar los datos de la zona DNS. Sin embargo, esta implementación presentó múltiples dificultades y problemas que finalmente llevaron a descartarla en favor de una estructura basada en `HashMap` anidados. Primero se explicará que RFC recomienda esta estructura y luego se detallarán los problemas encontrados.

4.5.1. RFC 1035

El RFC 1035, que define el estándar para el sistema de nombres de dominio, **sugiere** que los servidores DNS deberían utilizar una estructura de datos jerárquica para almacenar los registros de recursos (`RR`) de una zona DNS, pero también dice explícitamente que el manejo de estructuras internas es completamente libre. Citando textualmente la sección 6.1.2 del RFC 1035:

“While name server implementations are **free** to use any internal data structures they choose, the suggested structure consists of three major parts:

...

All of these data structures can be implemented an identical **tree structure format**”

De esta manera no se está rompiendo con ninguna norma o estándar al optar por una estructura diferente a la sugerida en el RFC.

4.5.2. Problemas de implementación

Implementar una estructura de árbol n-ario jerárquico para almacenar los registros DNS presentó varios desafíos técnicos y conceptuales que dificultaron su desarrollo y finalmente llevaron a su descarte. Algunos de los problemas más relevantes fueron:

- **Complejidad innata de la estructura:** La idea de un árbol es que vaya desde el dominio más general (raíz) hasta el más específico (hojas), pero para lograr esto desde el masterfile se debía agrupar los registros por niveles, lo que implicaba múltiples pasadas sobre los datos y una lógica compleja para manejar casos especiales como CNAMEs, delegaciones, etc.
- **Ownership de Rust:** Rust tiene un sistema de ownership muy estricto que complica la implementación de estructuras de datos recursivas como árboles. Manejar referencias mutables y compartidas en un árbol n-ario resultó ser particularmente desafiante, especialmente cuando se necesitaba modificar nodos en diferentes niveles del árbol. Además el sistema debía funcionar en un entorno concurrente, lo que añadió más restricciones.

Capítulo 5

Algoritmo de Búsqueda

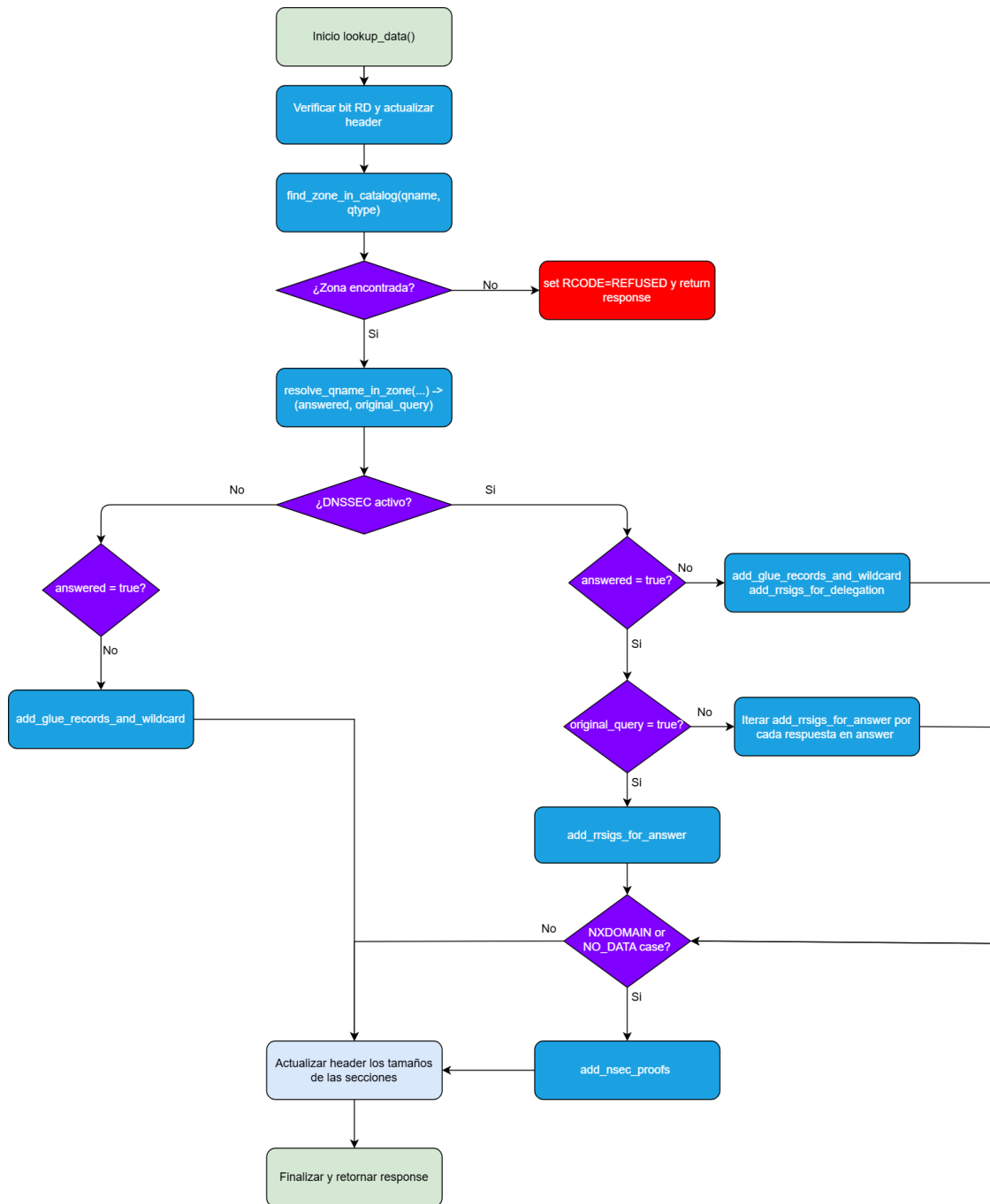
5.1. Flujo y funcionamiento

El algoritmo de búsqueda es el descrito en el RFC 1034, el cual se encarga recibir la consulta DNS y responder acorde a esta, buscando los registros en la base de datos del servidor. Este algoritmo contempla distintos escenarios, desde lo más general hasta lo más específico. Un resumen del mismo es el siguiente:

- Primero, se verifica si el dominio consultado pertenece al catálogo del servidor. Si no pertenece, se responde con el código **REFUSED**, indicando que el servidor no es autoritativo para dicho dominio.
 - Si el dominio consultado y el tipo de registro solicitado existen en la base de datos, se responde directamente con los registros encontrados.
 - Si el tipo de registro solicitado no es **CNAME**, pero existe un registro **CNAME** para el dominio, se realiza una nueva llamada a la función de búsqueda utilizando el valor del **CNAME** como nuevo nombre de dominio y conservando el tipo original. Los registros **CNAME** encontrados se agregan a la respuesta final.
- Si el proceso anterior no produce una respuesta, se continúa con la búsqueda de *glue records* y *wildcards*:
 - Si el dominio posee un *glue record* en la base de datos, se responde con dicho registro.
 - En caso contrario, se verifica si existe un *wildcard* aplicable al dominio consultado.
 - Si se encuentra un *wildcard*, se responde con los registros asociados a él.
 - Si no se encuentra ningún *wildcard*, se responde con **NXDOMAIN**, indicando que el dominio no existe.

Este proceso está simplificado, la versión de código es bastante más compleja y además cubre el estandar de DNSSEC. A continuación se mostrará un diagrama de flujo de la función

`lookup_data`, la cual es la función principal. Luego en otras subsecciones se explicará en detalle las funciones auxiliares.



La función `lookup_data` recibe como parámetros la consulta DNS completa y la bandera que indica si el modo DNSSEC está activado. A partir de la consulta, se extraen el nombre de dominio y el tipo de registro solicitado, y se inicia el proceso de búsqueda.

Primero, la función intenta localizar la zona correspondiente en el catálogo. Si la zona existe, se procede a resolver el nombre mediante la función `resolve_qname_in_zone`, la cual realiza la resolución directa o a través de registros internos. Esta función retorna dos valores booleanos:

- **answered**: Indica si la consulta fue respondida correctamente.
- **original_query**: Verdadero cuando la respuesta incluye registros del tipo solicitado originalmente, y falso cuando la respuesta se basa en un CNAME intermedio.

Si la consulta se realiza con DNSSEC habilitado, se añaden los registros de firma RRSIG correspondientes a los registros presentes en la respuesta. En caso de que no se haya podido responder, se agregan los **glue records**, posibles **wildcards** y sus respectivas firmas RRSIG. Cuando la respuesta no corresponde a la consulta original (es decir, proviene de CNAMEs), se añaden también las firmas RRSIG de dichos registros adicionales. Además, si la respuesta corresponde a un error NXDOMAIN o a un caso NO_DATA, se incluyen los registros NSEC que sirven como prueba criptográfica de inexistencia.

Por otro lado, si DNSSEC no está activo, el proceso se simplifica: si no se obtuvo una respuesta directa, se intenta completar la información añadiendo **glue records** o **wildcards**.

Finalmente, se actualizan los contadores y tamaños de las secciones en el encabezado (HEADER), y la función retorna la respuesta completa.

A continuación se explican cada una de las funciones auxiliares que componen el algoritmo de búsqueda.

5.1.1. `find_zone_in_catalog`

Esta función recibe el nombre de dominio consultado y el tipo de registro y busca por cada subdominio del dominio original si existe una zona en el catálogo que maneje ese dominio. Por ejemplo si el dominio consultado es `a.niclabs.cl`, la función buscará en el catálogo las zonas `a.niclabs.cl`, `niclabs.cl`, `cl` y `root`. Si encuentra una zona que maneje alguno de estos dominios, retorna la estructura de datos de la zona vista en la sección [REDACTED].

Esta función es bastante simple pero maneja casos especiales que son importantes y que vienen desde el estandar de DNSSEC. Por ejemplo si tipo consultado es DS, la función debe retornar la zona padre, ya que los registros DS son manejados siempre por la zona padre. Además existe un problema con el nombre de dominio raíz, ya que como se verá en una sección posterior, el nombre de dominio raíz se maneja internamente como `root` ya que no se puede usar `.` como nombre de carpeta en el sistema de archivos. Por lo tanto, si el dominio consultado es `.` la función debe buscar la zona `root` en el catálogo.

5.1.2. `resolve_qname_in_zone`

Esta función traduce en lógica programática el paso 3.a del algoritmo definido en el RFC 1034, gestionando de forma coherente la búsqueda interna, la detección de alias, las delegaciones y los casos sin datos dentro de una zona DNS.

Se puede dividir el funcionamiento de esta función en los siguientes pasos:

Presencia de un registro CNAME

Si el nombre consultado posee un registro CNAME, la función distingue dos situaciones:

- Si el tipo solicitado no es CNAME, el registro CNAME se añade igualmente a la respuesta, ya que informa que el nombre consultado es un alias. A continuación, el algoritmo sustituye el nombre original por el destino del CNAME y reinicia la búsqueda con ese nuevo nombre, tal como lo indica el RFC.
- Si el tipo solicitado es CNAME, se agregan los registros CNAME encontrados a la respuesta y se considera la consulta resuelta. En ambos casos, el bit de respuesta autoritativa (AA) se marca como verdadero, indicando que la información proviene directamente de la zona local.

Coincidencia exacta con nombre y tipo solicitado (distinto a CNAME)

Si el nombre posee registros del tipo consultado, estos se incorporan a la sección de respuestas. En este punto, la función evalúa si los registros corresponden a servidores de nombres (NS). Si el conjunto NS pertenece al apex de la zona (al nombre de la zona, por ejemplo `niclabs.cl`), la respuesta es autoritativa; si pertenece a un subdominio delegado, la función interpreta que se trata de una delegación y desactiva el bit de autoridad. De este modo, el comportamiento refleja la distinción entre datos autoritativos y delegaciones establecida por el RFC.

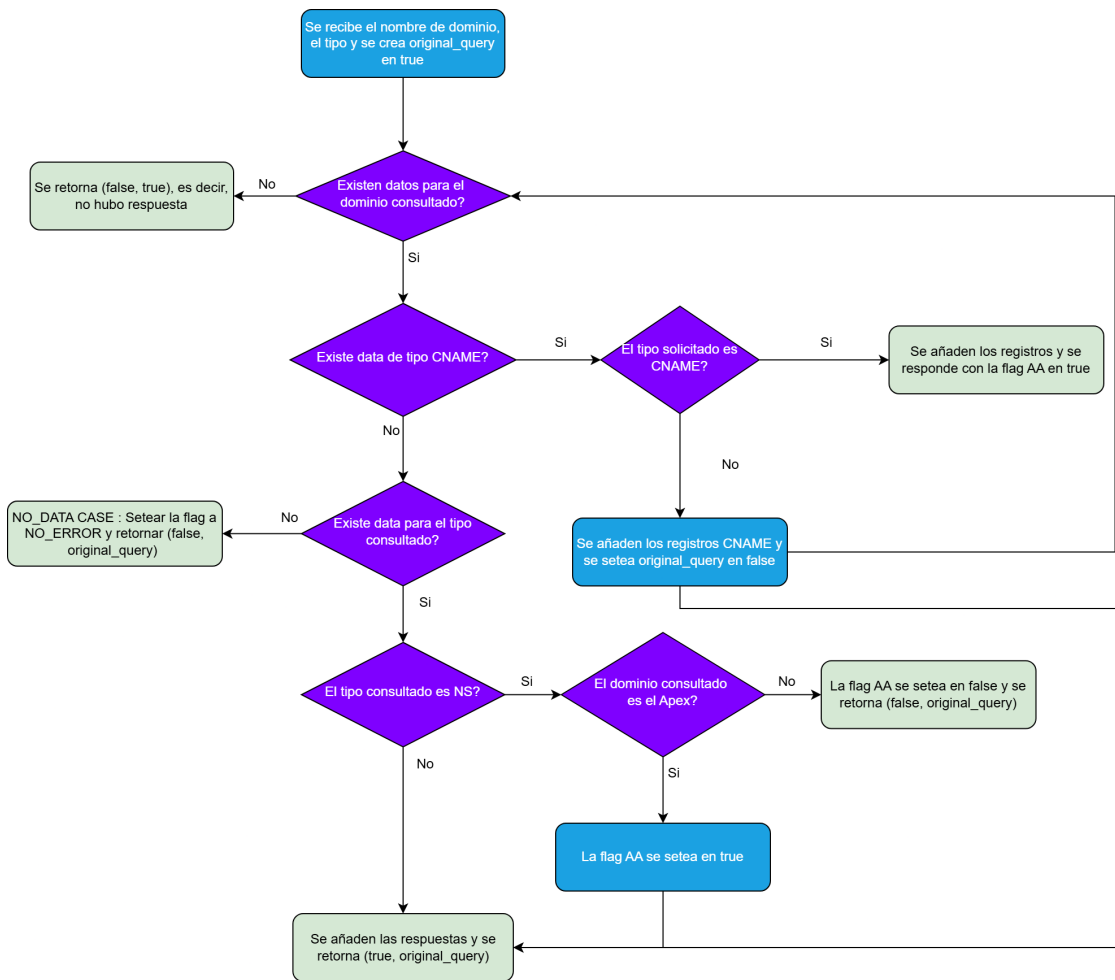
El nombre existe, pero no hay datos del tipo solicitado (NODATA)

Cuando el nombre se encuentra en la zona pero no existen registros del tipo pedido, la función retorna con un código de respuesta NOERROR, indicando que el nombre es válido pero no posee información del tipo solicitado. Este caso corresponde al escenario “no data” descrito en el RFC 1034, en el cual no se genera un error, sino simplemente una respuesta vacía.

El nombre no existe en la zona

Si el nombre de dominio no aparece en los datos de la zona, el algoritmo finaliza sin agregar información a la respuesta. La ausencia total de datos para ese nombre será tratada posteriormente por otra función que añade wildcards y/o NSECs según corresponda. Hasta este punto el código de respuesta debería ser `NXDOMAIN` pero no se asigna aún, ya que podría cambiar si se encuentran wildcards o glue records.

El diagrama de flujo de esta función es el siguiente:



5.1.3. add_glue_records_and_wildcard

La función `add_glue_records_and_wildcard` implementa las secciones 3.b y 3.c del algoritmo de búsqueda descrito en el RFC 1034. Su objetivo principal es manejar los casos de *delegations* (referrals) y de coincidencias con *wildcards*.

En primer lugar, la función analiza si el dominio consultado (`qname`) corresponde a un punto de delegación. Para ello, busca si existen registros **NS** asociados al nombre, y verifica que dicho conjunto de registros no esté acompañado por un **SOA**. Si existen registros **NS** sin **SOA**, significa que el dominio actual marca el inicio de una subzona delegada, por lo que el servidor debe responder con una *referral*. En este caso:

- Todos los registros **NS** encontrados se copian en la sección **Authority** de la respuesta.
- Luego, por cada nombre de servidor listado en esos registros **NS**, se busca si existe un registro **A** o **AAAA** correspondiente (denominado *glue record*). Si se encuentran, estos se añaden en la sección **Additional**.

Una vez construida la *referral*, el código establece el campo **Rcode** en **NOERROR** y marca la respuesta como no autoritativa (**AA** = 0), dado que el servidor no es responsable de la zona delegada.

Si no se encuentra un punto de delegación, el algoritmo continúa con la búsqueda de coincidencias mediante *wildcards*, correspondiente al paso 3.c del RFC 1034. Para ello, si existe un registro **RRset** asociado al *wildcard* y que coincida con el tipo de consulta (`qtype`), entonces:

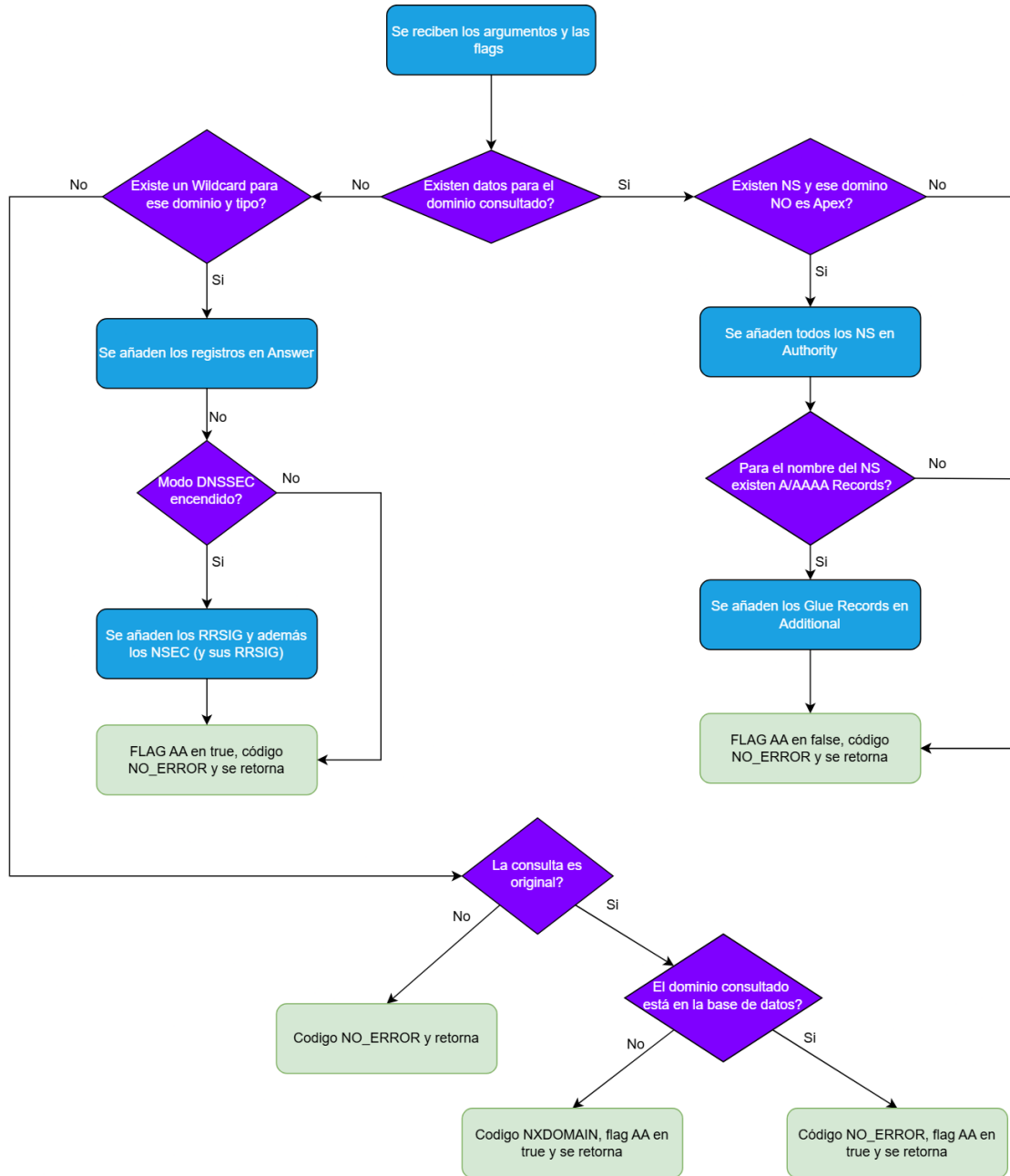
- Se añaden los registros encontrados a la sección **Answer**, utilizando como nombre propietario el `qname` original (según lo especificado por el RFC, aunque la firma se haya generado sobre el *wildcard*).
- Si existen firmas digitales **RRSIG** que cubran el tipo solicitado, estas se añaden igualmente en la sección **Answer**.
- Además, se incluyen en la sección **Authority** los registros **NSEC** y sus firmas **RRSIG** asociadas, cuando corresponda.

Finalmente, si no se encuentra ningún *wildcard* aplicable:

- Si la consulta actual corresponde al nombre original, el servidor determina si se trata de un caso **NO_DATA** (nombre existente sin registros del tipo solicitado). En tal caso, responde con **NOERROR** y **AA** = 1.
- En caso contrario, responde con **NXDOMAIN** (nombre inexistente) y también marca la respuesta como autoritativa.

Aclarar que **NO** se añaden los **RRSIG** en los glue record ya que la respuesta **NO** es autoritativa en ese caso.

El diagrama de flujo de la función descrita es el siguiente:



5.1.4. add_rrsigs_for_delegation

Esta función se encarga de añadir los registros DS y sus correspondientes firmas RRSIG a la sección de autoridad del mensaje DNS (**response**) cuando se genera una respuesta de delegación en un servidor DNSSEC.

En términos simples, cuando el servidor responde indicando que un dominio está delegado (por ejemplo, cuando responde con un conjunto de registros NS que apuntan al servidor autoritativo de una subzona), esta función busca si en su base de datos también existen los registros DS y sus firmas digitales (RRSIG) correspondientes, y los incluye en la respuesta.

El envío de los registros **DS** junto con una delegación está descrito en el RFC 4035, sección 2.3 (“Response Generation: Including DS and NSEC Records”). Aclarar también que esta función se ejecuta siempre independientemente de si la hubo o no un referral, ya que si no hay **DS** simplemente se omite su adición. Este aspecto podría ser mejorado en el futuro para evitar llamadas innecesarias a la función.

5.1.5. `add_rrsigs_for_answer`

Esta función se encarga de añadir los registros **RRSIG** correspondientes a un conjunto de registros de respuesta en la sección **Answer** del mensaje DNS (**response**).

En el contexto de DNSSEC, cada conjunto de registros (**RRset**) debe estar acompañado por su firma digital (**RRSIG**), que permite a los resolvers validar criptográficamente que los datos no han sido alterados.

La función solo añade los **RRSIG** correspondiente al tipo de consulta original, es decir, si la consulta fue por registros **A**, solo se añadirán los **RRSIG** que firman los registros **A**.

El comportamiento descrito está definido en el RFC 4035, sección 2.2 (“Response Generation: Including RRSIG Records”).

5.1.6. `add_nsecs_proofs`

Esta función se encarga de añadir las pruebas criptográficas de inexistencia (NSEC y RRSIG(NSEC)) a la sección Authority del mensaje DNS (**response**) cuando el servidor genera una respuesta de tipo `NODATA` o `NXDOMAIN` bajo DNSSEC.

Para dar contexto, los registros `NSEC` son registros especiales en DNSSEC que indican qué nombres de dominio y tipos de registros existen dentro de una zona firmada. Para hacer esto, cada registro `NSEC` enlaza un nombre de dominio con el siguiente en orden canónico, y contiene un mapa de bits que indica qué tipos de registros están presentes para ese dominio. Un orden canónico es un orden lexicográfico de los nombres en minúsculas.

Por ejemplo, si se tiene un registro `NSEC` que indica que el dominio `example.com` tiene registros `A` y `MX`, y enlaza con `foo.example.com`, esto significa que `example.com` existe y tiene esos tipos de registros, y que el siguiente dominio en orden canónico es `foo.example.com`, por lo tanto cualquier nombre entre `example.com` y `foo.example.com` no existe en la base de datos.

La función maneja los siguientes dos casos principales:

Caso 1: el dominio existe, pero el tipo de registro no (respuesta `NODATA`)

Si el dominio consultado está presente en la base de datos, pero no tiene registros del tipo solicitado (**qtype**), entonces se deben incluir los registros `NSEC` y `RRSIG(NSEC)` asociados a ese mismo dominio en la sección Authority.

Estos registros demuestran que, aunque el nombre existe, no hay ningún registro del tipo solicitado (por ejemplo, no existe un `MX` para ese dominio). Esto se hace a través del mapa de tipos de registro contenidos en el registro `NSEC`, que indica qué tipos de registros están presentes para ese dominio.

Caso 2: el dominio no existe (respuesta `NXDOMAIN`)

Si el dominio consultado no está en la base de datos, se necesita saber cual registro `NSEC` cubre el rango que incluye el nombre inexistente.

Para ello, se realiza una búsqueda binaria en la lista de nombres `NSEC`:

- Si el dominio es menor que todos los nombres existentes, se utiliza el último nombre de la lista (por la naturaleza circular del espacio de nombres DNSSEC).
- En caso contrario, se toma el nombre inmediatamente anterior al dominio consultado.

Aclarar que la comparación entre dominios se realiza en su forma canónica.

Luego, se insertan en la respuesta los registros **NSEC** y **RRSIG(NSEC)** del propietario determinado. El uso de los registros **NSEC** para demostrar la inexistencia de nombres o tipos de registros está definido en el RFC 4035, sección 3.1.3 (“Authenticated Denial of Existence”).

Está sería la última función explicada del algoritmo de búsqueda. Con esto se cierra el capítulo.

A continuación se mostrarán respuestas del algoritmo para distintos casos de prueba en la sección [REDACTED].

5.2. Resultados según cada caso de consulta

A continuación se muestran ejemplos de respuestas generadas por el algoritmo de búsqueda para distintos casos de consulta. Cada ejemplo incluye los records presentes en la base de datos, la consulta realizada y la respuesta generada en el formato del comando `dig`.

Aclarar que la sección `Additional` no se añadirá en los ejemplos a menos que contenga registros, para simplificar la visualización.

5.2.1. Consulta con coincidencia exacta con `rtype` y el nombre de dominio

Nombre de dominio	Clase	Tipo	Data
www.example.com.	IN	A	0.0.0.0
www.example.com.	IN	A	0.0.0.1

Al preguntar directamente por la IP de **www.example.com** se encuentra una coincidencia exacta con el nombre y el tipo solicitado en los datos, por lo que la respuesta DNS tendrá todos los *RRs* del *RRset* en la sección **Answer**, además de los *RRSIG* correspondientes:

```
1 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4660
2 ;; flags: qr aa; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1
3
4 ;; QUESTION SECTION:
5 ;www.example.com.          IN      A
6
7 ;; ANSWER SECTION:
8 www.example.com.  3600    IN      A      192.0.2.1
9 www.example.com.  3600    IN      A      192.0.2.2
10 www.example.com.  3600    IN      RRSIG   A 8 3 3600 0 0 12786 www.
    example.com. (keydata ...)
```

5.2.2. Consulta con coincidencia solo en el nombre de dominio, pero existe un registro `CNAME`

Nombre de dominio	Clase	Tipo	Data
blog.example.com	IN	CNAME	blog2.example.com.
blog2.example.com.	IN	A	1.2.3.4

Al consultar por **blog.example.com** el algoritmo encuentra un registro de tipo `CNAME` y lo guarda en la sección **Answer**. Luego, ejecuta nuevamente el algoritmo utilizando el nombre de dominio especificado en el `CNAME`, logrando así encontrar el registro con la dirección IP correspondiente.

```

1 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4660
2 ;; flags: qr aa; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1
3
4 ;; QUESTION SECTION:
5 ;blog.example.com.          IN      A
6
7 ;; ANSWER SECTION:
8 blog.example.com.  3600    IN      CNAME    blog2.example.com.
9 blog2.example.com. 3600    IN      A        1.2.3.4
10 blog.example.com.  3600    IN      RRSIG    CNAME 8 3 3600 0 0 12786
        blog.example.com. (keydata ...)
11 blog2.example.com. 3600    IN      RRSIG    A 8 3 3600 0 0 12786
        blog2.example.com. (keydata ...)

```

5.2.3. Consulta que sale de la zona autoritativa : Referral con Glue Records

Nombre de dominio	Clase	Tipo	Data
b.example.com	IN	NS	ns.a.b.example.com.
ns.a.b.example.com.	IN	A	4.3.2.1

Al consultar por la IP de **b.example.com**, se encuentra un registro de tipo **NS**. Al comprobar que no existe ningún registro **SOA** en ese dominio (lo que indica que el servidor no es autoritativo en este caso), se debe guardar el registro **NS** en la sección **Authority**. Además, se debe verificar si existe un registro *glue* correspondiente y, de ser así, añadirlo en la sección **Additional**.

Además se añade el registro **DS** y su firma **RRSIG** correspondiente en la sección **Authority**, esto ya que se está delegando a una subzona. También se añade el registro **NSEC** para confirmar la inexistencia del registro **A** solicitado a través del mapa de bits.

```

1 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4660
2 ;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 5, ADDITIONAL: 2
3
4 ;; QUESTION SECTION:
5 ;b.example.com.          IN      A
6
7 ;; AUTHORITY SECTION:
8 b.example.com.  3600    IN      NS        ns.a.b.example.com.
9 b.example.com.  3600    IN      DS        12345 8 2 (digest_data
        ...)
10 b.example.com.  3600    IN      RRSIG    DS 8 3 3600 0 0 12786 b.
        example.com. ( keydata ... )
11 b.example.com.  3600    IN      NSEC     ns1.example.com. NS DS
        NSEC

```

```

12 b.example.com.      3600      IN      RRSIG      NSEC 8 3 3600 0 0 12786
    example.com. ( keydata ... )
13
14 ;; ADDITIONAL SECTION:
15 .                  32768      IN      OPT      4096
16 ns.a.b.example.com. 3600      IN      A      4.3.2.1

```

5.2.4. Consulta sin coincidencia en el nombre de dominio pero existe un wildcard

Nombre de dominio	Clase	Tipo	Data
*.example.com.	IN	A	0.0.0.0

Al consultar por cualquier dominio que no exista en los datos pero sea subdominio de **example.com**, como por ejemplo **unknown.example.com**, se usará el wildcard y se guardará la respuesta en la sección **Answer**. Además se añaden los registros **NSEC** y **RRSIG(NSEC)** correspondientes en la sección **Authority** para probar la inexistencia del nombre consultado.

```

1 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4660
2 ;; flags: qr aa; QUERY: 1, ANSWER: 2, AUTHORITY: 2, ADDITIONAL: 1
3
4 ;; QUESTION SECTION:
5 ;unknown.example.com.      IN      A
6
7 ;; ANSWER SECTION:
8 unknown.example.com.      3600      IN      A      0.0.0.0
9 unknown.example.com.      3600      IN      RRSIG      A 8 2 3600 0 0 12786
    example.com. ( keydata ... )
10
11 ;; AUTHORITY SECTION:
12 *.example.com.      3600      IN      NSEC      b.example.com. A RRSIG
    NSEC
13 *.example.com.      3600      IN      RRSIG      NSEC 8 3 3600 0 0 12786
    example.com. ( keydata ... )

```

5.2.5. Consulta sin coincidencia en nombre de dominio y sin wildcard: NXDOMAIN

Acá no se muestra lo que hay en la base de datos, ya que el cliente consultará por la IP de un dominio que no existe (**unknown.example.com**) y no hay wildcard que aplique. Para lograr esta respuesta se borró temporalmente el wildcard de los datos.

```

1 ;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 4660
2 ;; flags: qr aa; QUERY: 1, ANSWER: 0, AUTHORITY: 2, ADDITIONAL: 1
3
4 ;; QUESTION SECTION:
5 ;unknown.example.com.      IN      A
6
7 ;; AUTHORITY SECTION:
8 example.com.      3600      IN      NSEC      b.example.com. NS SOA
   RRSIG NSEC DNSKEY
9 example.com.      3600      IN      RRSIG      NSEC 8 2 3600 0 0 12786
   example.com. ( keydata ... )

```

En la respuesta en formato DIG no se ve pero el rcode es NXDOMAIN.

5.2.6. Consulta sin coincidencia en el tipo y sin wilcard: NODATA

Nombre de dominio	Clase	Tipo	Data
www.example.com.	IN	A	1.2.3.4

Al consultar por un tipo que no existe para un dominio que sí existe, como por ejemplo pedir un registro MX para **www.example.com**, se responde con NODATA y se añaden los registros NSEC y RRSIG(NSEC) correspondientes en la sección *Authority* para probar la inexistencia del tipo solicitado a través del mapa de tipos.

```

1 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4660
2 ;; flags: qr aa; QUERY: 1, ANSWER: 0, AUTHORITY: 2, ADDITIONAL: 1
3
4 ;; QUESTION SECTION:
5 ;www.example.com.      IN      MX
6
7 ;; AUTHORITY SECTION:
8 www.example.com.      3600      IN      NSEC      blog.example.com. A
   RRSIG NSEC
9 www.example.com.      3600      IN      RRSIG      NSEC 8 3 3600 0 0 12786
   example.com. ( keydata ... )

```

El código de respuesta es NOERROR pero no hay registros en la sección *Answer*, siendo esto equivalente a un caso de NO_DATA.

Capítulo 6

EDNS0

Extension DNS 0 (EDNS0) es una extensión del protocolo DNS que permite superar algunas de las limitaciones del formato original del DNS, como el tamaño máximo de los mensajes y la capacidad de transportar información adicional. Fue definido en el RFC 2671 y posteriormente actualizado por el RFC 6891. EDNS0 es un requisito para muchas de las funcionalidades avanzadas del DNS, incluyendo DNSSEC.

6.1. Características principales de EDNS0

Para habilitar EDNS0, los clientes y servidores DNS incluyen un registro especial en sus mensajes DNS llamado `OPT` (Option), el cual va en la sección Additional. Este Resource Record (RR) no usa los campos de manera tradicional, sino que los utiliza para transportar información adicional sobre las capacidades y opciones del DNS. Específicamente, cada campo se utiliza de la siguiente manera:

- **NAME:** Siempre es un nombre vacío (0 bytes).
- **TYPE:** Siempre es 41, que indica que es un registro `OPT`.
- **CLASS:** Indica el tamaño máximo de mensaje que el emisor/cliente puede manejar. Por ejemplo, un valor de 4096 bytes permite mensajes DNS más grandes que el límite original de 512 bytes.
- **TTL:** Se utiliza para transportar varias opciones y banderas.
 - **Extended RCODE** : Los primeros 8 bits representan el RCODE extendido.
 - **EDNS Version** : 8 bits que representan la versión de EDNS0 (actualmente siempre es 0).
 - **Z field** : 16 bits reservados para banderas, específicamente el bit 15 es la flag `DO` (DNSSEC OK).

La flag más importante es el DO (DNSSEC OK), que indica si el emisor puede manejar respuestas DNSSEC.

- **RDLENGTH**: Indica la longitud de los datos de opciones adicionales.
- **RDATA**: Contiene las opciones adicionales que pueden ser utilizadas para diversas funcionalidades, como la negociación de capacidades, pero no se utiliza para DNSSEC.

6.2. Ejemplo de mensaje DNS con EDNS0

Para facilitar el proceso de construcción de una consulta con EDNS0 se creó una función especial llamada `create_dns_opt_query`, la cual recibe como parámetros el nombre de dominio, el tipo de consulta y retorna una consulta DNS que incluye un registro OPT en la sección Additional, usando como tamaño máximo de mensaje 4096 bytes.

Un ejemplo del registro OPT generado por esta función es el siguiente:

```
1 [ResourceRecord { name: DomainName { name: '.'}, rtype: OPT,
  rclass: UNKNOWN(4096), ttl: 32768, rdlength: 0, rdata: OPT(
    OptRdata { option: [] }) }]
```

En este ejemplo, se puede observar que el campo `rclass` tiene un valor de UNKNOWN con un 4096 dentro, indicando que el emisor puede manejar mensajes DNS de hasta 4096 bytes. Además se puede ver que el campo `ttl` tiene un valor de 32768, lo que indica que la flag DO (DNSSEC OK) está activada

```
1 TTL (32 bits): 00000000 00000000 10000000 00000000
2 |<-RCODE->|<-VERS->|<-----Z----->|
```


Capítulo 7

DNSSEC

En este capítulo se verá cómo funciona DNSSEC (Domain Name System Security Extensions) dentro del servidor, desde la generación de las claves criptográficas, pasando por distintos procesos de firmas para cada **RRSET**, hasta la creación e inserción de los registros **NSEC** y **DS**.

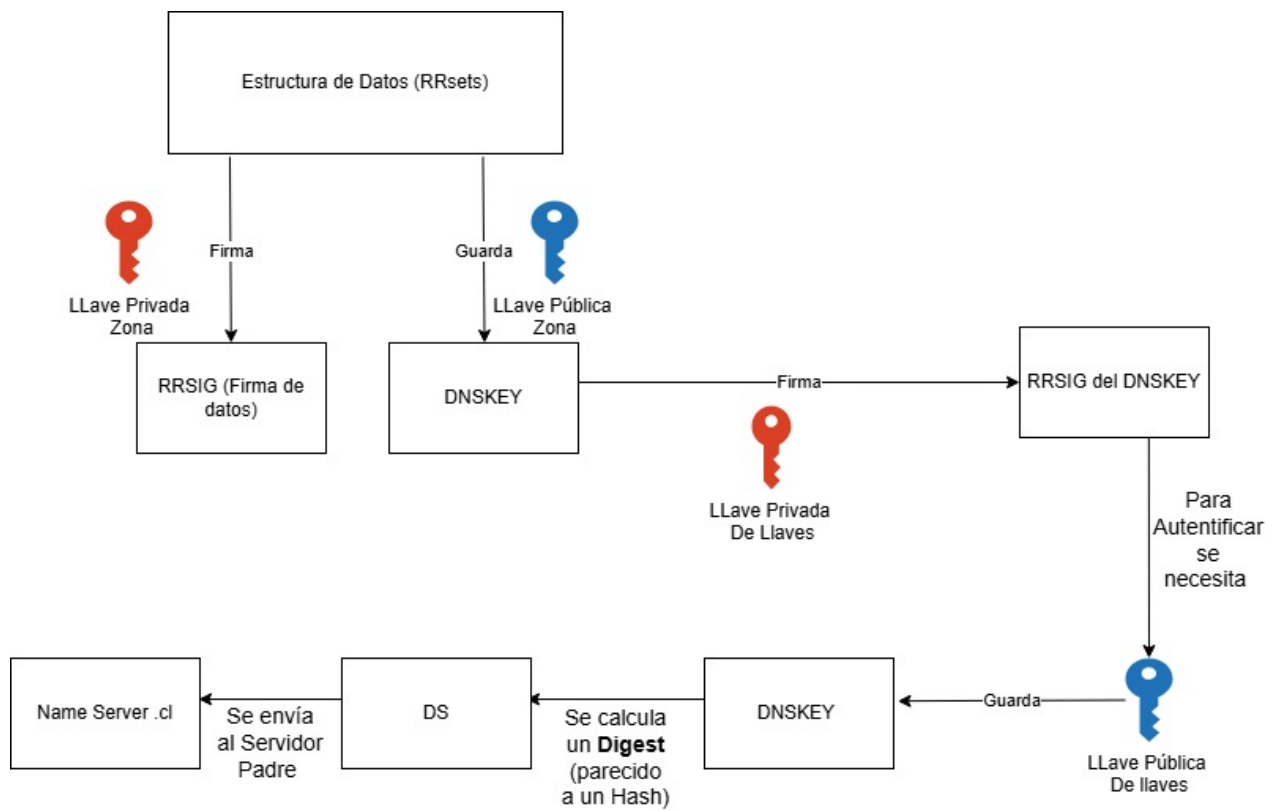
Pero para poder comprobar que las respuestas DNS son auténticas y válidas, es necesario tener un cliente que pueda validar las respuestas DNSSEC, específicamente usando la cadena de autenticación. En primera instancia, esta verificación se iba a realizar dentro del **stub-resolver** de la librería **dns-rust**, pero debido a que las funciones de parsing de la librería están rotas (las funciones **to_bytes** y **from_bytes**), se decidió hacer todo de manera local. En un capítulo posterior se explicará por qué las funciones de parsing están rotas y cómo se podría solucionar.

7.1. Servidor

7.1.1. Generación de llaves criptográficas

El proceso de firma es complejo y requiere de dos pares de llaves criptográficas por cada zona: un par de llaves para firmar los datos (llaves **ZSK** : Zone Signing Key) y otro par de llaves para firmar las llaves **ZSK** (llaves **KSK** : Key Signing Key). Esto es requerido ya que DNSSEC utiliza un modelo de confianza basado en la jerarquía del DNS, donde cada zona firma sus registros **DNSKEY** con su llave **KSK**, y la zona padre guarda un registro **DS** que apunta a la llave **KSK** de la zona hija, firmando este registro con su propia llave **KSK**. De esta manera, se puede construir una cadena de confianza desde la raíz del DNS hasta cualquier zona, verificando las firmas en cada nivel.

Para mayor claridad se muestra el siguiente diagrama que ilustra el proceso de firmas en DNSSEC:



El proceso de generación de las llaves criptográficas del sistema se realiza mediante el algoritmo RSA, utilizando la librería `rsa` del lenguaje Rust. Como este proceso debe hacerse una sola vez (y no cada vez que se ejecuta el servidor) se separó la lógica dentro de un archivo aparte llamado `generate_keys.rs`, archivo que se encuentra dentro del directorio `src/bin`, para que así pueda ser ejecutado de manera independiente.

El proceso de generación de las llaves criptográficas se realiza de la siguiente manera:

- Se tiene una lista de zonas de los cuales se quieren generar las llaves criptográficas. En este caso, los dominios son `example.com`, `niclabs.cl`, `.com`, `.cl` y `root` (raíz).
- Por cada dominio, se generan los dos pares de llaves criptográficas (KSK y ZSK) utilizando la función `generate_keypair`
- Cada llave generada se guardan archivos dentro del directorio `keys/zona` a través de la función `write_keys_to_disk`

`generate_keypair`

En primer lugar, la función `generate_keypair(bits)` recibe como parámetro el tamaño de la llave en bits (por ejemplo, 1024 o 2048). Dentro de la función, se instancia un generador de números aleatorios seguro del sistema operativo (`OsRng`), el cual provee la entropía

necesaria para la generación de claves criptográficamente seguras, a partir de este generador se crea una llave privada RSA

Posteriormente, la llave pública se deriva directamente desde la llave privada usando el método `to_public_key()`. El resultado de esta función es una tupla que contiene ambas llaves: `(RsaPrivateKey, RsaPublicKey)`.

write_keys_to_disk

La función `write_keys_to_disk(domain, zsk, ksk)` se encarga de almacenar físicamente las llaves generadas en disco. Esta función recibe como parámetros el nombre de la zona (por ejemplo, `c1`) y los pares de llaves correspondientes al ZSK (Zone Signing Key) y KSK (Key Signing Key).

Dentro de la función, primero se crea un directorio con la estructura `keys/domain` para asegurar que exista la carpeta donde se guardarán los archivos PEM (Privacy Enhanced Mail), formato comúnmente utilizado para almacenar llaves criptográficas. Posteriormente, se escriben las llaves en formato PKCS#8 (estándar de representación de llaves en formato PEM) mediante los métodos `to_pkcs8_pem()` para las llaves privadas y `to_public_key_pem()` para las llaves públicas.

Cada llave se almacena en un archivo independiente con nombres descriptivos:

- `zsk_priv.pem`
- `zsk_pub.pem`
- `ksk_priv.pem`
- `ksk_pub.pem`

Aclarar que un nombre de carpeta NO puede ser "."(punto), por lo menos en Windows, ya que genera un error al intentar crear el directorio, esto obliga a renombrar la carpeta de la raíz a `root`

7.1.2. Estructuras de datos para las llaves

Una vez generadas las llaves y guardadas en formato PEM dentro de la carpeta `keys/zona` correspondiente, es necesario cargarlas en memoria a través de alguna estructura global accesible desde cualquier parte del servidor. Para esto, se definieron las siguientes estructuras de datos

ZoneKeyStore

La estructura `ZoneKeyStore` es un mapa hash que asocia cada zona (representada como un string) con un puntero a su correspondiente conjunto de llaves criptográficas, que a su vez es otra estructura llamada `ZoneKeys`. Esta estructura se define de la siguiente manera:

```
1 pub struct ZoneKeyStore {
2     keys: HashMap<String, Arc<ZoneKeys>>,
3 }
```

Se usa `Arc` (Atomic Reference Counting) para permitir que múltiples partes del programa puedan compartir acceso a las llaves de una zona sin necesidad de clonaras.

La variable global accesible desde cualquier parte del servidor se define como:

```
1 pub static GLOBAL_ZONE_KEY_STORE: Lazy<ZoneKeyStore> = Lazy::new
2 (|| {
3     let mut store = ZoneKeyStore::new();
4
5     let domains = ["example.com.", "niclabs.cl.", "com.", "cl.",
6 "root."];
7
8     for domain in &domains {
9         let path = format!("keys/{}", domain);
10        let zone_keys = ZoneKeys::load_from_path(&path);
11        store.insert_zone_keys(domain.to_string(), Arc::new(
12zone_keys));
13    }
14
15    store
16});
```

La cual basicamente itera sobre la lista de zonas, carga las llaves desde el disco usando la función `ZoneKeys::load_from_path` y las inserta en el `ZoneKeyStore`.

ZoneKeys

La estructura `ZoneKeys` almacena las cuatro llaves criptográficas necesarias para una zona DNSSEC: la llave privada y pública del ZSK (Zone Signing Key) y la llave privada y pública del KSK (Key Signing Key). Esta estructura se define de la siguiente manera:

```
1 pub struct ZoneKeys {
2     zsk_priv: RsaPrivateKey,
3     zsk_pub: RsaPublicKey,
4     ksk_priv: RsaPrivateKey,
5     ksk_pub: RsaPublicKey,
6 }
```

Su metodo `load_from_path(path)` se encarga de leer los archivos PEM desde el disco, parsearlos con el método `from_pkcs8_pem` de la librería `rsa` y crear una instancia de `ZoneKeys` con las llaves correspondientes.

7.1.3. Inserción de DNSKEY's RRSETS en la estructura de datos

Una vez que ya se tienen las llaves cargadas en memoria, se pueden utilizar para crear los registros `DNSKEY` necesarios para cada zona. Estos registros contienen las llaves públicas utilizadas para verificar las firmas de los registros DNS, tanto del par ZSK como del par KSK.

Por lo que se debe insertar un `RRSET` de tipo `DNSKEY` por cada estructura de datos de cada zona. Esto se realiza en la función `insert_dnskey_rrsets`.

El algoritmo `insert_dnskey_rrset` crea un `RRset` para la zona con clase `IN`, tipo `DNSKEY` y el TTL dado por el proceso de parsing del masterfile. Luego obtiene las claves desde la variable global `GLOBAL_ZONE_KEY_STORE`, codifica las claves públicas con `encode_rsa_public_key` y construye dos `DnskeyRdata`, los cuales tienen los siguientes campos:

- **flags:** Indica el tipo de llave. El valor 256 indica una llave ZSK (Zone Signing Key) y el valor 257 indica una llave KSK (Key Signing Key). En este caso se usan ambos valores.
- **protocol:** Siempre es 3, según lo especificado en el RFC 4034.
- **algorithm:** Indica el algoritmo criptográfico utilizado. En este caso, se usa el valor 8, que corresponde a RSA/SHA-256.
- **public_key:** Contiene la representación codificada de la llave pública en formato wire.

En cuanto al proceso de codificación de las llaves públicas RSA en el formato requerido por DNSSEC, se utiliza la función `encode_rsa_public_key`, la cual se encarga de codificar una clave pública RSA en el formato wire definido por el RFC 3110.

Primero obtiene los bytes del exponente (**e**) y del módulo (**n**) en orden big-endian. Luego, si el exponente tiene una longitud entre 1 y 255 octetos, este se representa con un solo byte; si supera los 255 bytes, se antepone un byte con valor 0 seguido de dos bytes que indican su longitud.

Después de codificar la longitud, se concatenan los bytes del exponente y del módulo, formando la representación final que sigue la sintaxis del campo `public_key` en los registros `DNSKEY`

Finalmente se insertan los datos a las estructuras correspondientes.

7.1.4. Inserción de los RRSIG de los DNSKEY's RRSET

Una vez insertados los registros `DNSKEY` en la estructura de datos, se procede a firmarlos utilizando la llave `KSK` (Key Signing Key), conforme a lo establecido en el RFC 4035, Sección 2.1.1. Esta firma es distinta de las que se aplican a los demás `RRsets` de la zona, los cuales se firman con la llave `ZSK` (Zone Signing Key). La función `insert_rrsig_rrset_dnskey` es la encargada de realizar este proceso.

En primer lugar, el algoritmo obtiene el conjunto de registros `DNSKEY` desde la estructura de datos y recupera la llave privada `KSK` almacenada en `GLOBAL_ZONE_KEY_STORE`. Luego, calcula el `key_tag` de la `KSK` aplicando el algoritmo definido en el Apéndice B del RFC 4034, el cual consiste en sumar los bytes del `RDATA` del registro `DNSKEY` en formato wire y reducir el resultado a 16 bits. Este valor actúa como un identificador único de la clave utilizada en la firma.

Posteriormente, se construye la estructura `RRSIG_Rdata`, completando los campos definidos en el RFC 4034, Sección 3.1. Cada uno de estos campos cumple una función específica dentro del registro de firma:

- **type_covered**: indica el tipo de registro DNS cubierto por la firma. En este caso, su valor es `DNSKEY`, ya que la firma se aplica sobre el conjunto de claves públicas del dominio.
- **algorithm**: especifica el algoritmo criptográfico utilizado para generar la firma. Se asigna el valor 8, correspondiente a `RSASHA256`, conforme al RFC 4034.
- **labels**: representa la cantidad de etiquetas (labels) del nombre de dominio al que pertenece el `RRSet`. Su valor se obtiene mediante la función auxiliar `count_labels(zone_name)`.
- **original_ttl**: contiene el valor TTL original del `RRSet` firmado, copiado directamente desde el conjunto `DNSKEY`.
- **signature_inception** y **signature_expiration**: definen el intervalo de validez de la firma. En esta implementación se establecen con valor 0, ya que el servidor no implementa aún manejo de tiempo real ni soporte para zonas dinámicas.
- **key_tag**: identifica de forma única la clave usada para la firma. En este caso se utiliza el valor calculado previamente a partir de la `KSK`.
- **signer_name**: indica el nombre del propietario de la clave utilizada para la firma, correspondiente al dominio de la zona (`zone_name`).
- **signature**: almacena la firma criptográfica generada sobre el `RRSet` de `DNSKEY`. A continuación se detalla el proceso para calcular este campo.

Para generar la **signature** se necesita firmar el **TBS** (To Be Signed), el cual sería la concatenación de los campos del **RRSIG** (exceptuando la misma **signature**) con el `DNSKEY` en versión canónica. Este proceso se realiza mediante la función auxiliar `build_tbs_data` siguiendo el orden establecido por el proceso de canonicalización de `DNSSEC`.

Finalmente, se utiliza la llave privada del par KSK para firmar el TBS utilizando el algoritmo RSA con SHA-256b y se añade el RRSET de RRSIG resultante a la estructura de datos del servidor.

7.1.5. Creación e inserción de firmas y el manejo de casos bordes

Después de insertar y firmar los registros DNSKEY, el siguiente paso consiste en firmar todos los demás RRSET presentes en la zona. Naturalmente, los registros RRSIG no deben firmarse, ya que ellos mismos contienen las firmas de los conjuntos de registros.

Este procedimiento es prácticamente idéntico al proceso de firma de los DNSKEY, con la diferencia de que en este caso se utiliza la llave ZSK (Zone Signing Key) en lugar de la KSK (Key Signing Key), y se aplica a cada RRSET perteneciente a un dominio de la zona. Sin embargo, existen ciertos casos borde que deben manejarse cuidadosamente: los registros NSEC, los registros no autoritativos o de delegación, y los **wildcards**.

En primer lugar, el proceso de generación y firma de los registros NSEC es más extenso y se detalla en la siguiente sección. En cuanto a los registros no autoritativos o de delegación (por ejemplo, los registros NS que apuntan a una zona hija), estos no deben ser firmados, ya que son responsabilidad de la zona delegada. Para detectar estos casos dentro de la estructura de datos, se implementó la función `is_authoritative_rrset`, cuyo flujo general es el siguiente:

- Verifica que el nombre del RRset pertenezca efectivamente a la zona. Si no está dentro de la zona, no se considera autoritativo.
- Evita firmar los RRset de delegación, es decir, los registros NS que no se encuentran en el apex de la zona.
- Evita firmar los **glue records**, es decir, registros A o AAAA asociados a una delegación, los cuales solo sirven para resolver los nombres de los servidores delegados.
- En cualquier otro caso, el RRset se considera autoritativo y debe ser firmado.

En el caso de los wildcards, su identificación es directa, ya que el nombre del RRset comienza con un asterisco (*). Cuando esto ocurre, se debe restar uno al conteo de etiquetas (**labels**) del campo correspondiente en el RRSIG. Esto permite que el resolver pueda reconocer que la respuesta proviene de un wildcard y validar correctamente su firma. De no hacerlo, el cliente no podría distinguir el wildcard de un nombre explícito y la validación DNSSEC fallaría.

Esta regla está establecida explícitamente en el RFC 4035, Sección 5.3.2, que indica:

```
1 "If the number of labels in an RRset's owner name is greater than the
   Labels field of the covering RRSIG RR,
2 then the RRset and its covering RRSIG RR were created as a result of
   wildcard expansion."
```

7.1.6. Creación e inserción de los registros NSEC

Los registros NSEC (Next Secure) son fundamentales en DNSSEC para proporcionar autenticación de ausencia de datos. Estos contienen los siguientes campos:

- **next_name**: Indica el siguiente nombre de dominio en orden canónico dentro de la zona.
- **type_bitmap**: Es un mapa de bits que señala los tipos de registros presentes para el nombre de dominio actual.

La función `create_and_insert_nsec_rrsets` construye e inserta todos los NSEC RRSET de la zona, siguiendo estrictamente lo especificado por los RFC 4034 y 4035. El proceso completo involucra dos tareas principales:

Construcción de la lista de nombres para NSEC (NSEC owner names)

El primer paso consiste en determinar exactamente cuáles nombres del archivo de zona deben poseer un NSEC. Esto se realiza mediante la función `gather_nsec_names`, la cual recorre toda la estructura interna de datos y selecciona únicamente aquellos nombres que corresponden a información autoritativa o a puntos de delegación. Esta filtración es esencial porque los registros NSEC describen únicamente nodos válidos dentro de la jerarquía DNS de la zona firmada. Una vez reunidos estos nombres, la lista se ordena en el orden canónico, siguiendo las reglas del RFC 4034 Appendix B.

Construcción de los mapas de tipos

Posteriormente, para cada nombre de esta lista se construye el campo `type_bit_maps`. Durante la iteración de los tipos presentes en un nombre, la implementación evita incluir todos los tipos considerados “pseudo-tipos”, tales como OPT, TSIG, ANY, entre otros. Estos tipos no forman parte de la información firmada ni pueden aparecer legítimamente en un bitmap de un NSEC, tal como lo establece el RFC 4034 4.1.2. Luego, dependiendo del tipo de nodo, existen reglas especiales. Si el nombre es un punto de delegación (es decir, contiene un RRset NS pero no es el apex), solo se deben incluir en el bitmap los tipos NS y DS. Esto se debe a que, según el RFC 4035 2.3, el servidor padre no es autoritativo para el resto de los registros del dominio delegado y, por lo tanto, no debe afirmar su existencia mediante un NSEC. En contraste, si el nombre corresponde a información completamente autoritativa, se incluyen todos los tipos válidos presentes en el nodo. En ambos casos, de manera obligatoria, se agrega también el tipo NSEC al bitmap, ya que cada nombre que tiene un NSEC necesariamente debe anunciarlo explícitamente, como lo dicta el RFC 4034 4.1.2.

Finalmente, con el nombre actual, el siguiente nombre en orden canónico y el bitmap resultante, se construye la RData NSEC. El TTL del registro se toma desde el campo “minimum TTL” del SOA del apex, siguiendo la recomendación del RFC 4034 2.3.4. Con toda esta información se produce un nuevo RRset de tipo NSEC, el cual se inserta en la estructura de datos de la zona.

7.1.7. Inserción de RRSIGs para los NSEC RRSET

El proceso de construcción de los registros NSEC es, en esencia, el mismo que el descrito previamente en la subsección **Creación e inserción de firmas y el manejo de casos bordes**. Sin embargo, surge una pregunta natural: ¿por qué las firmas RRSIG correspondientes a los NSEC se generan en un paso separado, en lugar de ser firmadas junto con el resto de los RRSET? La razón es estructural. Los registros NSEC deben reflejar con precisión todos los tipos existentes en cada nombre, y eso incluye los RRSIG. En otras palabras, el mapa de tipos (`type_bit_maps`) de un NSEC debe considerar la presencia de las firmas ya generadas. Por este motivo, los RRSIG de los demás RRSET deben existir antes de crear los NSEC. Si se intentara firmar todo en un solo paso, los NSEC no podrían incluir los tipos correctos, produciendo un conjunto inconsistente e inválido bajo DNSSEC. Debido a esta dependencia lógica, la firma de los NSEC se ejecuta necesariamente como una fase posterior y separada.

7.1.8. Creación del RRSET DS para la zona padre

Un registro DS (Delegation Signer) es un tipo especial de registro DNS que se utiliza en DNSSEC para establecer una cadena de confianza entre una zona padre y una zona hija. El cual contiene los siguientes campos:

- **key_tag**: Identificador único de la clave pública de la zona hija.
- **algorithm**: Algoritmo criptográfico utilizado por la clave pública.
- **digest_type**: Tipo de función hash utilizada para generar el digest.
- **digest**: Resultado de aplicar la función hash a la concatenación del nombre canonico de la zona hija y su llave pública.

La construcción del RRSET DS se realiza en la función `build_ds_rrset`. Para ello primero obtiene el DNSKEY RRSET del apex de la zona y prepara un nuevo RRSET DS con el mismo nombre, clase y TTL. Luego, recorre cada registro DNSKEY y selecciona únicamente aquellos que corresponden a la clave KSK. Para cada llave calcula el **key tag** y genera el valor del **digest** siguiendo las especificaciones del RFC 4509, concatenando el nombre de dominio en forma canónica con el RDATA del DNSKEY y aplicando la función hash dada según el **digest_type**. Con esta información se construye la estructura `DsRdata`, que posteriormente se inserta como registro dentro del DS RRSET. Finalmente, el RRSET resultante se retorna para que pueda ser insertado en la estructura de datos de la zona padre.

El servidor no tiene acceso directo a la zona padre, por lo que al generar el RRSET DS, este se imprime en consola, se copian los datos y se insertan manualmente en el master-file de la zona superior. Esto coincide con la práctica actual de los servidores DNSSEC, ya que no existe todavía un mecanismo estandarizado y ampliamente adoptado para que los registros DS se envíen automáticamente al progenitor.

En la reunión ICANN 84 (Dublín, 25-30 de octubre de 2025) se abordó precisamente este tema en el contexto del taller “Towards an Industry Best Practice for DS Automation” presentado por Peter Thomassen (deSEC), donde se revisaron estadísticas de adopción y los desafíos operativos para la automatización del envío de DS.

7.2. Resolver

Es fundamental verificar que las firmas generadas por el servidor sean correctas dentro del contexto de DNSSEC. Para ello, se implementó completamente la lógica de validación de firmas y de la cadena de autenticación. Originalmente, este proceso iba a realizarse dentro del `stub-resolver` de la librería `dns-rust`; sin embargo, debido a problemas en sus funciones internas de parsing, se optó por efectuar toda la validación localmente, replicando con precisión el flujo que tendría un `stub-resolver` real.

El proceso de autenticación debe distinguir entre varios escenarios posibles en una respuesta DNSSEC:

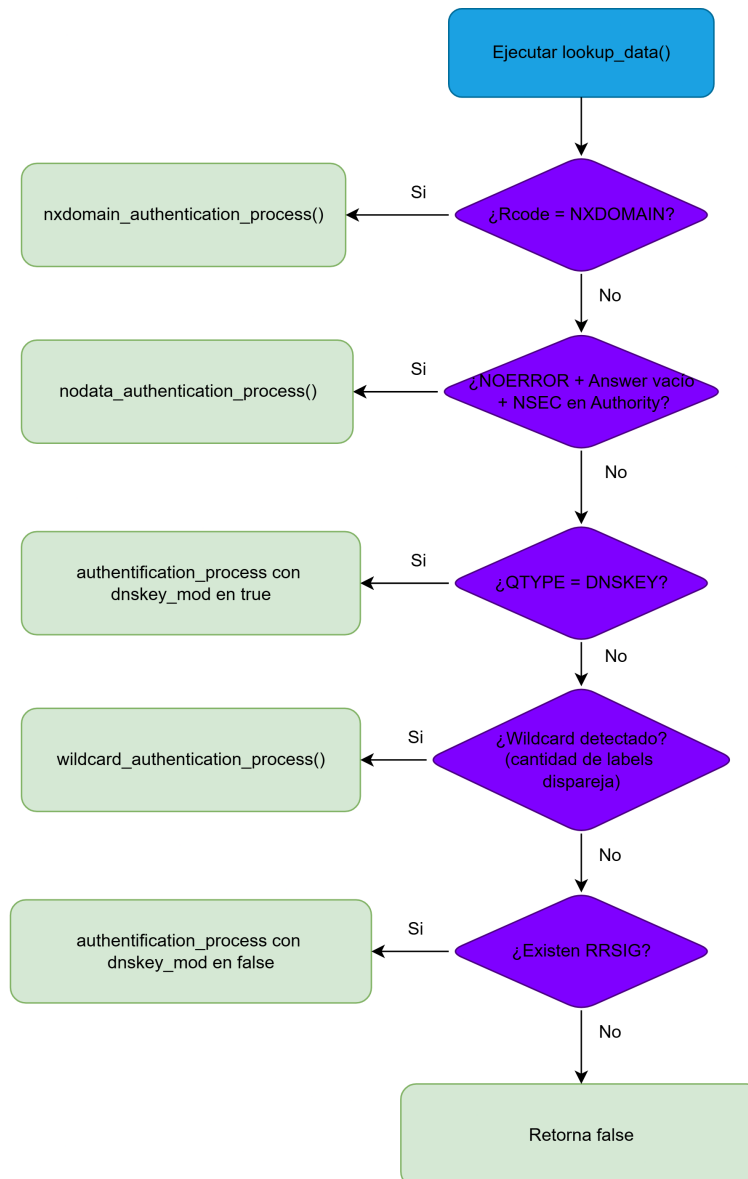
- Respuestas con datos en la sección `Answer` y sus `RRSIG` asociados.
- Respuestas con código `NXDOMAIN`.
- Respuestas `NO_DATA`.
- Respuestas originadas por consultas `DNSKEY`.
- Respuestas generadas a partir de un `Wildcard`.

Para gestionar correctamente cada caso, se definió la función `dnssec_authentication`, la cual recibe la consulta DNS original y retorna `true` si la respuesta es válida bajo DNSSEC, o `false` en caso contrario. El flujo general de esta función es el siguiente:

- En primer lugar, se ejecuta la función `lookup_data` con `DNSSEC` habilitado, obteniéndose así la respuesta completa que será inspeccionada para determinar qué mecanismo de validación corresponde aplicar.
- Si el código de respuesta en el `Header` es `NXDOMAIN`, significa que el nombre consultado no existe en la zona. En este caso se activa el proceso de autenticación basado en pruebas NSEC de inexistencia de dominio, mediante `nxdomain_authentication_process`.
- Si el código es `NOERROR`, pero la sección `Answer` está vacía y la sección `Authority` contiene un registro `NSEC`, la respuesta corresponde al caso `NO_DATA`. Para validar esta situación, se ejecuta `nodata_authentication_process`, el cual verifica la ausencia del tipo consultado mediante la prueba NSEC apropiada.
- Si la consulta original corresponde al tipo `DNSKEY`, se activa el modo especial de autenticación que utiliza la `KSK`. En este caso se invoca `authentication_process` con la bandera `dnskey_mod` en `true`.

- Si no se cumplen las condiciones anteriores, se analiza la presencia de registros **RRSIG** en las secciones **Answer** y **Authority**. Luego, se compara la cantidad de labels (etiquetas) del nombre de dominio del RRset con el campo **labels** contenido en la firma **RRSIG**. Si el número de labels del dominio es mayor que el valor en el **RRSIG**, se trata de una respuesta producida por **wildcard expansion**. En este caso, se ejecuta **wildcard_authentication_process**.
- Si no se detecta wildcard, pero al menos un **RRSIG** existe, la respuesta corresponde a un caso de validación estándar, por lo que se invoca **authentication_process**.
- Finalmente, si la respuesta no contiene ningún **RRSIG**, no existe información criptográfica que validar. El proceso falla explícitamente retornando **false**, pues la respuesta no cumple los requisitos de seguridad DNSSEC.

El siguiente diagrama de flujo ilustra visualmente este proceso:



A continuación se describen todos los procesos de validación para cada caso:

7.2.1. Proceso de autenticación para nombres inexistentes (NXDOMAIN)

La función `nxdomain_authentication_process` se encarga de validar respuestas con código `NXDOMAIN`, asegurando que el nombre consultado no existe en la zona, utilizando los registros `NSEC` presentes en la sección `Authority` de la respuesta DNS.

La función tiene dos verificaciones principales:

Verificación del intervalo NSEC

Itera sobre cada registro `NSEC` en la sección `Authority` y, para cada uno, verifica si el nombre consultado cae dentro del intervalo definido por el nombre dueño del `NSEC` y su `next_name`. Esta verificación se realiza comparando los nombres en su forma canónica. La función utilizada para esto, `name_in_nsec_range`, la cual implementa exactamente la lógica descrita en el RFC 4034 6.2.1: comprueba si el nombre consultado está estrictamente entre el `owner` y el `next`, o si se trata del caso especial de “wrap-around”, donde el `next` es lexicográficamente menor que el `owner` y el intervalo cruza el final del espacio de nombres. Si el nombre consultado está cubierto por alguno de los intervalos `NSEC`, se continúa retorna verdadero; si ninguno lo cubre, la autenticación retorna falso.

Para ilustrar el caso normal, considere:

`example.com. NSEC foo.example.com.`

Este `NSEC` cubre cualquier nombre estrictamente entre `example.com.` y `foo.example.com.`. Por ejemplo, `bar.example.com.` está dentro del intervalo, retornando verdadero, en cambio `baz.example.com.` no lo está, retornando falso.

En contraste, el caso de `wrap-around` ocurre cuando `next_name` es lexicográficamente menor que `owner`. Por ejemplo:

`z.example.com. NSEC example.com.`

Aquí el intervalo envuelve el final del espacio de nombres, cubriendo todos los nombres mayores que `z.example.com.` (como `zzz.example.com.`) y también todos los menores que `example.com.` (como `a.example.com.`). Si el nombre consultado cae dentro de alguno de estos intervalos `NSEC`, se retorna `true`, si ninguno lo cubre (como en el caso de `m.example.com.`), se retorna `false`.

Verificación de firmas RRSIG del NSEC

Luego se verifican las firmas RRSIG asociadas a los NSEC encontrados, estas firmas están en la sección **Authority** y el proceso normal-común es que estén en la sección **Answer**, por lo que se limpia la sección **Answer** y se copian las firmas RRSIG a esta sección para reutilizar la función de autenticación estándar la cual es `authentication_process`. De esta manera, si la firma es válida, se confirma que el NSEC es auténtico y por ende el nombre consultado no existe en la zona.

Si ambas verificaciones son exitosas, la función retorna `true`, confirmando que la respuesta NXDOMAIN es válida bajo DNSSEC.

7.2.2. Proceso de autenticación para respuestas NO_DATA

La función `nodata_authentication_process` valida respuestas DNS que indican la ausencia de datos para un tipo específico asociado a un nombre existente, es decir, respuestas **NO_DATA**. Este caso es similar al proceso de autenticación para **NXDOMAIN**, con la diferencia de que aquí el nombre consultado *sí existe* en la zona, pero el tipo solicitado **no**.

El proceso de validación consta de 4 pasos principales:

Verificación de existencia del nombre

Primero se debe encontrar un registro NSEC cuyo **owner name** coincida exactamente con el nombre consultado. Si no se encuentra tal NSEC, la autenticación falla inmediatamente, retornando `false`, ya que esto indicaría que el nombre no existe en la zona.

Verificación de la ausencia del tipo solicitado

Una vez localizado el NSEC correspondiente al nombre consultado, se examina su campo `type_bitmap` para verificar que el tipo solicitado no esté presente. Si el tipo está ausente en el bitmap, se sigue con el proceso, de lo contrario se retorna `false`.

Adición del DS asociado si es que existe

Un caso particular que siempre se manifiesta como **NO_DATA** es el de los **delegation points**. En una delegación, la zona padre es autoritativa únicamente para los registros **NS** (y, si existe, **DS**), mientras que **no** es autoritativa para ningún otro tipo perteneciente a la zona hija. Por lo tanto, cualquier consulta hacia el padre que solicite un tipo distinto de **NS** o **DS** debe producir necesariamente una respuesta **NO_DATA**. El nombre existe, pero la zona padre no puede servir el tipo solicitado porque no le pertenece.

En estos casos, el RFC 4035, Sección 2.3, establece que la zona padre *debería* (“**SHOULD**”) incluir un registro **DS** si este está disponible. El propósito de este **SHOULD** es permitir a los validadores confirmar la continuidad de la cadena de confianza hacia la zona delegada. Es decir, si se entrega un **NSEC** para probar la ausencia de un tipo determinado y, adicionalmente, un **DS** (con su correspondiente **RRSIG**) para demostrar que la delegación está correctamente firmada, el validador puede:

- Confirmar que no existen datos del tipo consultado en la zona padre.
- Verificar que la delegación hacia la zona hija mantiene una cadena de confianza válida.

Esta lógica de añadir el **DS** en estos casos de **delegation points** está cubierta por el algoritmo de búsqueda, específicamente en la función auxiliar `add_rrsigs_for_delegation`, que tiene un mal nombre ya que no solo añade los **RRSIG** sino que también añade el **DS** si es que existe. Esto se vió en la sección [REDACTED]

Para cumplir con esta recomendación, la función inspecciona la sección **Authority** en busca de registros **DS** asociados al nombre consultado y, en caso de encontrarlos, los conserva para su validación posterior. Asimismo, verifica que los registros **RRSIG** que cubren a dichos **DS** también estén presentes en la respuesta, asegurando así que cualquier información entregada sobre la delegación pueda ser autenticada criptográficamente.

Verificación de firmas **RRSIG** del **NSEC** y **DS**

Finalmente, se verifican las firmas **RRSIG** asociadas tanto al registro **NSEC** como, cuando corresponde, al registro **DS**. El procedimiento es análogo al utilizado en la autenticación de respuestas **NXDOMAIN**: se limpia la sección **Answer** y se trasladan a ella los registros **NSEC**, **DS** y sus respectivas firmas **RRSIG**. Esto permite reutilizar directamente la función estándar de validación `authentication_process`, garantizando así que todas las pruebas criptográficas asociadas al caso **NO_DATA** se verifiquen de manera uniforme y consistente.

7.2.3. Autenticación para respuestas provenientes de Wildcards

La función `wildcard_authentication_process` se encarga de validar respuestas DNS que han sido generadas mediante la expansión de un **wildcard** (comodín). Este tipo de respuestas ocurre cuando un nombre no existente coincide con un patrón del tipo `*.example.com`, y el servidor autoritativo genera los registros utilizando dicho **wildcard**, tal como lo establecen los RFC 4592 y RFC 4035.

El proceso de validación consta de los siguientes pasos:

Descubrir cuál fue el wildcard utilizado

El primer paso consiste en identificar el nombre del wildcard original que fue utilizado para generar la respuesta. Para ello, la función inspecciona la sección **Authority** en busca del primer registro **NSEC**, ya que el RFC 4035 requiere que toda respuesta basada en **wildcard** incluya un registro **NSEC** que pruebe la inexistencia explícita del nombre consultado.

En una respuesta genuina de wildcard, el dueño de ese **NSEC** es precisamente el nombre del wildcard (por ejemplo, ***.example.com.**). Este valor es crucial, ya que es el **nombre real** sobre el que se generó y firmó el **RRSIG** del **RRset** en el servidor autoritativo.

Reconstruir la sección Answer reemplazando el nombre consultado por el wildcard

La sección **Answer** que llega al resolver contiene como **owner name** el nombre consultado por el cliente, por ejemplo **unknown.example.com.**, incluso si dicho nombre no existe. Esto es un comportamiento obligatorio de acuerdo con RFC 1034 4.3.3, el cual establece que:

the name server creates RRs with an owner name equal to the query name and contents taken from the wildcard RRs

Sin embargo el **RRSIG** del **RRset** no fue generado sobre **unknown.example.com.** sino sobre el wildcard ***.example.com.** Por lo tanto, para poder verificar correctamente la firma, es necesario reconstruir localmente el **RRset tal como fue firmado originalmente**. Esto implica tomar cada registro de la sección **Answer** y reemplazar su nombre dueño por el wildcard detectado.

El resultado es una nueva sección **Answer** que refleja el estado real del **RRset** al momento de la firma en el servidor.

Autenticar las firmas del RRset reconstruido

Una vez reconstruida la sección **Answer**, se procede a verificar el **RRSIG** asociado mediante la función estándar **authentication_process**. Si esta verificación falla, la respuesta completa debe considerarse inválida, ya que implicaría que el **RRset** entregado no corresponde al wildcard que lo generó, o que la firma ha sido alterada.

Validar los NSEC asociados

Finalmente, la función verifica que la expansión del wildcard se haya producido legalmente y que el nombre consultado realmente **no exista en la zona**. Para esto:

- Se crea una copia del mensaje de respuesta.

- Se limpian las secciones **Answer** y **Additional**, dejando únicamente la sección **Authority**. Dicha sección contiene los **NSEC** y sus correspondientes **RRSIG**, que son los elementos necesarios para demostrar la no existencia del nombre consultado.
- Se ajusta el código de respuesta a **NXDOMAIN**, ya que ahora se está validando la inexistencia del nombre original.

El motivo de este paso está normado: el RFC 4035 3.1.3 exige que una respuesta basada en **wildcard** debe incluir pruebas criptográficas que demuestren la inexistencia del nombre exacto consultado, lo cual se logra mediante un **NSEC** que cubra dicho nombre.

La función `nxdomain_authentication_process` valida entonces esta copia modificada del mensaje.

Si todas las verificaciones son exitosas, la función retorna **true**, confirmando que la respuesta generada por el **wildcard** es válida bajo DNSSEC.

7.2.4. Proceso de Autenticación DNSSEC Estándar

La función `authentication_process` implementa la validación estándar de firmas DNSSEC para respuestas que contienen datos en la sección **Answer**. De hecho, todas las funciones anteriores de validación (**NXDOMAIN**, **NO_DATA** y **Wildcards**) terminan invocando esta función para validar las firmas **RRSIG** correspondientes. La función toma como argumentos la respuesta DNS completa y una bandera booleana `dnskey_mod` que indica si la consulta original fue de tipo **DNSKEY**. El proceso de validación consta de las siguientes etapas:

Preparación de información y variables iniciales

El algoritmo comienza extrayendo desde la respuesta DNS las secciones **Answer**, **Authority** y **Question**, clasificando los registros en dos grupos:

- **Registros de datos:** Todos los registros que no son **RRSIG**, los cuales contienen la información real solicitada.
- **Registros RRSIG:** Registros **RRSIG**, que contienen las firmas digitales.

A partir del nombre de dominio consultado se determinan las dos últimas etiquetas, que constituyen el apex de la zona (por ejemplo, **example.com.** o **niclabs.cl.**) Este apex es fundamental para obtener las claves **DNSKEY** necesarias en las siguientes etapas.

Obtención de las claves **DNSKEY** y sus firmas (**DNSKEY RRset**)

Para poder validar una respuesta DNSSEC, el resolver debe obtener el conjunto de claves **DNSKEY** del apex. Esto lo realiza la función auxiliar `get_pubkeys_dnskey_and_rrsig`, que:

- Crea una consulta DNS de tipo `DNSKEY` dirigida al apex de la zona y la resuelve utilizando la función `lookup_data` con DNSSEC habilitado.
- Identifica y separa las claves públicas según su rol criptográfico: la `ZSK` (Zone Signing Key) y la `KSK` (Key Signing Key).
- Convierte ambas claves a estructuras `RSA` utilizables por el validador.

La función retorna: el par (`ZSK`, `KSK`), el `RRset DNSKEY` y el `RRset RRSIG(DNSKEY)`

Verificación de las firmas de la respuesta

La verificación de firmas sobre los datos recibidos (`Answer + Authority`) se realiza mediante la función `verify_rrsig`. Para cada `RRSIG` se ejecutan los siguientes pasos:

- Determinar el tipo de `RRset` que la firma cubre (`type_covered`).
- Reunir el `RRset` correspondiente desde los registros recibidos.
- Construir el `RRset` en formato canónico DNSSEC.
- Construir la estructura `TBS` (To-Be-Signed), que combina la metada del `RRSIG` y el `RRset` canónico.
- Verificar la firma usando Hash: `SHA-256`, Algoritmo: `RSA PKCS1 v1.5` y clave pública: `ZSK` (modo normal) o `KSK` (si `dnskey_mod` está activo)

Notar que en estricto rigor se debería usar el Hash y Algoritmo indicados en el campo `algorithm` del `RRSIG`, pero en esta implementación solo se soporta `RSA-SHA256`, ya que es el algoritmo más comúnmente utilizado en la práctica y fue el único implementado en el servidor.

Verificación del `RRset DNSKEY`

Cuando `dnskey_mod` está deshabilitado (modo normal), el algoritmo debe además validar el propio conjunto `DNSKEY`. Esta verificación es crítica, porque el `RRset DNSKEY` es la base criptográfica que permite validar al resto de los `RRsets` de la zona. El proceso consiste en:

- Tomar el `RRset DNSKEY`, su `RRSIG(DNSKEY)` y construir la estructura `TBS` correspondiente.
- Verificar la firma usando exclusivamente la `KSK`, ya que solo la `KSK` puede firmar el `RRset DNSKEY`.

Solo si esta firma es válida, las claves `DNSKEY` pueden considerarse auténticas y confiables para validar el resto de la zona.

Cadena de Autenticación DNSSEC

Para validar completamente la cadena de confianza DNSSEC, el algoritmo necesita:

- Zona hija: el apex (`example.com` o `niclabs.cl`)
- Zona padre: la última etiqueta del apex (`com`, `cl`)
- El `DnskeyRdata` de la KSK de la zona hija, ya que el DS en la zona padre debe corresponder con ese valor.

Con los datos anteriores, delega la validación de la cadena de autenticación a la función especial `verify_ds_chain`. La cual se verá en el siguiente sección ya que es un proceso complejo que involucra múltiples pasos y consultas.

Resultado final

La respuesta DNS se considera válida únicamente si todas las validaciones anteriores fueron exitosas: las firmas de los `RRsets`, la firma del `RRset DNSKEY` y la cadena de autenticación. Si alguna de estas verificaciones falla, la función retorna `false`, indicando que la respuesta no cumple con los requisitos de seguridad DNSSEC

7.2.5. Proceso de verificación de la cadena de autenticación

La validación de la cadena de autenticación DNSSEC es el mecanismo que permite verificar que la clave pública (KSK) utilizada por una zona hija es auténtica y se encuentra correctamente delegada por su zona padre y así sucesivamente hasta llegar a una autoridad de confianza (la zona `root`). Este proceso es fundamental para garantizar la integridad y autenticidad de las respuestas DNS, ya que asegura que los datos no han sido alterados y provienen de una fuente confiable, obligando que la zona padre respalde la autenticidad de la zona hija.

La función es explicada en las siguientes etapas:

Obtención del registro DS en la zona padre

El procedimiento comienza realizando una consulta DNS de tipo `DS`, utilizando como nombre objetivo el apex de la zona hija. Esta consulta debe ser resuelta por la zona padre, ya que los registros `DS` siempre se almacenan en la delegación que el padre mantiene respecto del hijo.

Es importante señalar que la función `lookup` utilizada para enviar esta consulta incorpora un manejo especial para este caso. Tal como se explicó en la sección correspondiente del

algoritmo de resolución, si `lookup` tratara esta consulta como una consulta DNS normal, intentaría resolverla dentro de la propia zona hija (pues el nombre coincide con su apex), lo que impediría encontrar el registro `DS` en la zona correcta. Para evitar este error, `lookup` detecta consultas de tipo `DS` y fuerza la resolución hacia la zona padre. Aclarar que este comportamiento especial es debido a que no se habla con otros servidores DNS externos, sino que todo se simula localmente.

Como resultado de esta etapa, se obtiene desde la zona padre: El `RRset DS` y su correspondiente `RRset RRSIG(DS)`.

Obtención del DNSKEY RRset y sus firmas en la zona padre

Una vez extraído el `DS`, el verificador debe obtener los registros `DNSKEY` de la zona padre junto con sus firmas `RRSIG(DNSKEY)` y el par de llaves públicas (`ZSK`, `KSK`) de dicha zona. Esto se realiza mediante la función auxiliar `get_pubkeys_dnskey_and_rrsig`.

Esta información es necesaria para verificar tanto las firmas del `DS` como la autenticidad del propio `DNSKEY` del padre.

Verificación criptográfica en la zona padre

Con toda la información disponible, se realizan dos verificaciones fundamentales:

- **Verificación del RRset DNSKEY del padre con su propia KSK:** El `RRset DNSKEY` debe estar firmado con la `KSK` de la zona padre. Si esta firma no es válida, no se puede confiar en ninguna clave de esa zona, por lo que toda la cadena debe considerarse inválida.
- **Verificación de la firma RRset DS utilizando la ZSK del padre:** El `RRset DS` se encuentra firmado por la `ZSK` del padre. Si esta verificación falla, el `RRset DS` tampoco es confiable, lo que impide autenticar el vínculo entre el padre y el hijo.

Ambas validaciones se realizan mediante la función `verify_rrsig`.

Verificación de la correspondencia entre el DS y la KSK de la zona hija

Si ambas firmas del paso anterior son válidas, el algoritmo procede a verificar que alguno de los registros `DS` obtenidos en la zona padre corresponde efectivamente con la clave `KSK` de la zona hija.

Para ello se itera por cada registro `DS` y se compara su `digest` con el valor calculado a partir de la concatenación del nombre canónico del apex de la zona hija y el `RData` del `DNSKEY KSK` de la zona hija, utilizando la función hash indicada en el campo `digest_type` del `DS`. Este calculo es el indicado en el RFC [REDACTED]

Aclarar también que en DNSSEC, el uso del **key tag** para identificar rápidamente la clave correcta es un **SHOULD**, no un **MUST**, esto según el RFC [REDACTED], por lo que este diseño no depende de él, y en cambio comprueba directamente que los **digest** coincidan

Si algún DS coincide, se confirma que la zona padre efectivamente delega y certifica la clave KSK de la hija.

Ascenso recursivo hasta la raíz

Una vez validado el vínculo entre el padre y el hijo, se debe continuar verificando la cadena de autenticación hacia arriba. El algoritmo actúa así:

- Si la zona padre actual corresponde a la zona raíz, la verificación concluye exitosamente, ya que la raíz constituye el trust anchor y no necesita validación adicional, además no se puede seguir subiendo ya que la zona raíz no tiene padre.
- En caso contrario:
 - La zona padre se convierte ahora en la zona hija del siguiente nivel
 - Se determina cuál es el nuevo nombre de la zona padre
 - Se extrae el **DnskeyRdata** correspondiente (que será utilizada en el próximo cálculo de DS)
 - El proceso completo se repite recursivamente sobre ese nivel superior.

Resultado final

La cadena DS se considera válida únicamente si todas las etapas se completan exitosamente:

- El DS se obtuvo correctamente desde la zona padre.
- El **DNSKEY RRset** y el **DS RRset** del padre fueron firmados y verificados correctamente.
- Algún registro DS coincide con el **digest** calculado a partir de la KSK de la zona hija.
- La verificación se mantiene válida en cada nivel superior hasta alcanzar la zona raíz.

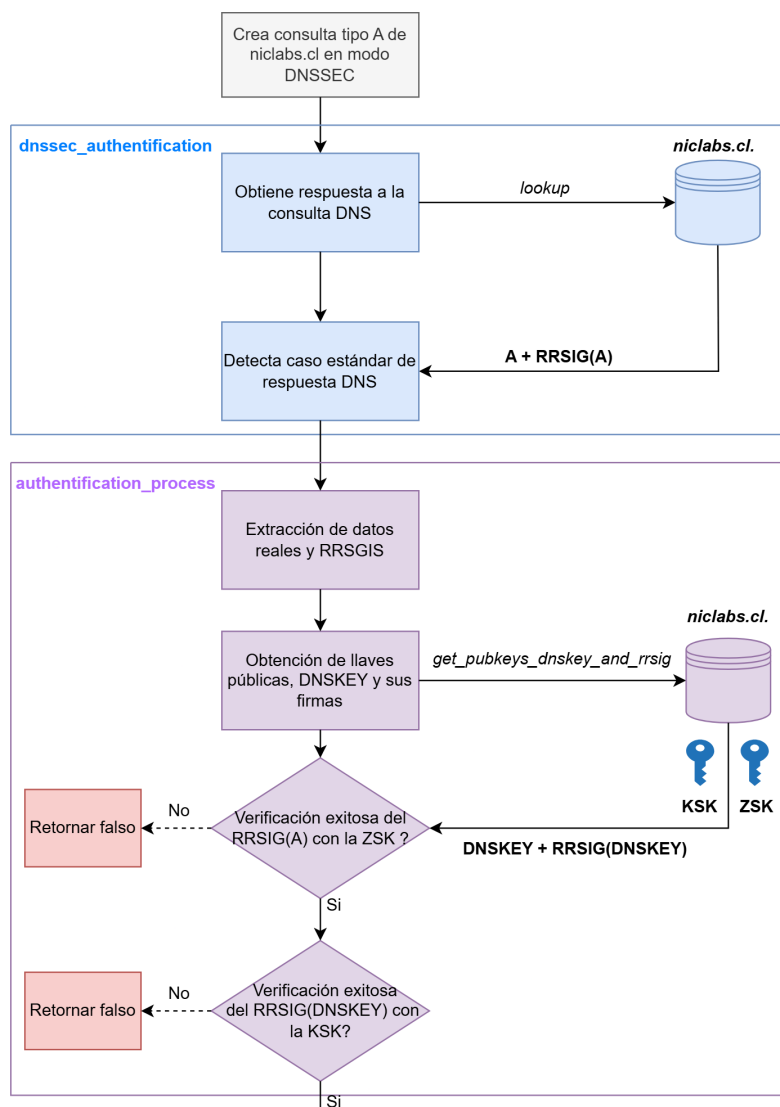
Solo entonces la clave KSK de la zona inicial puede considerarse auténtica y correctamente anclada en la cadena de confianza de DNSSEC.

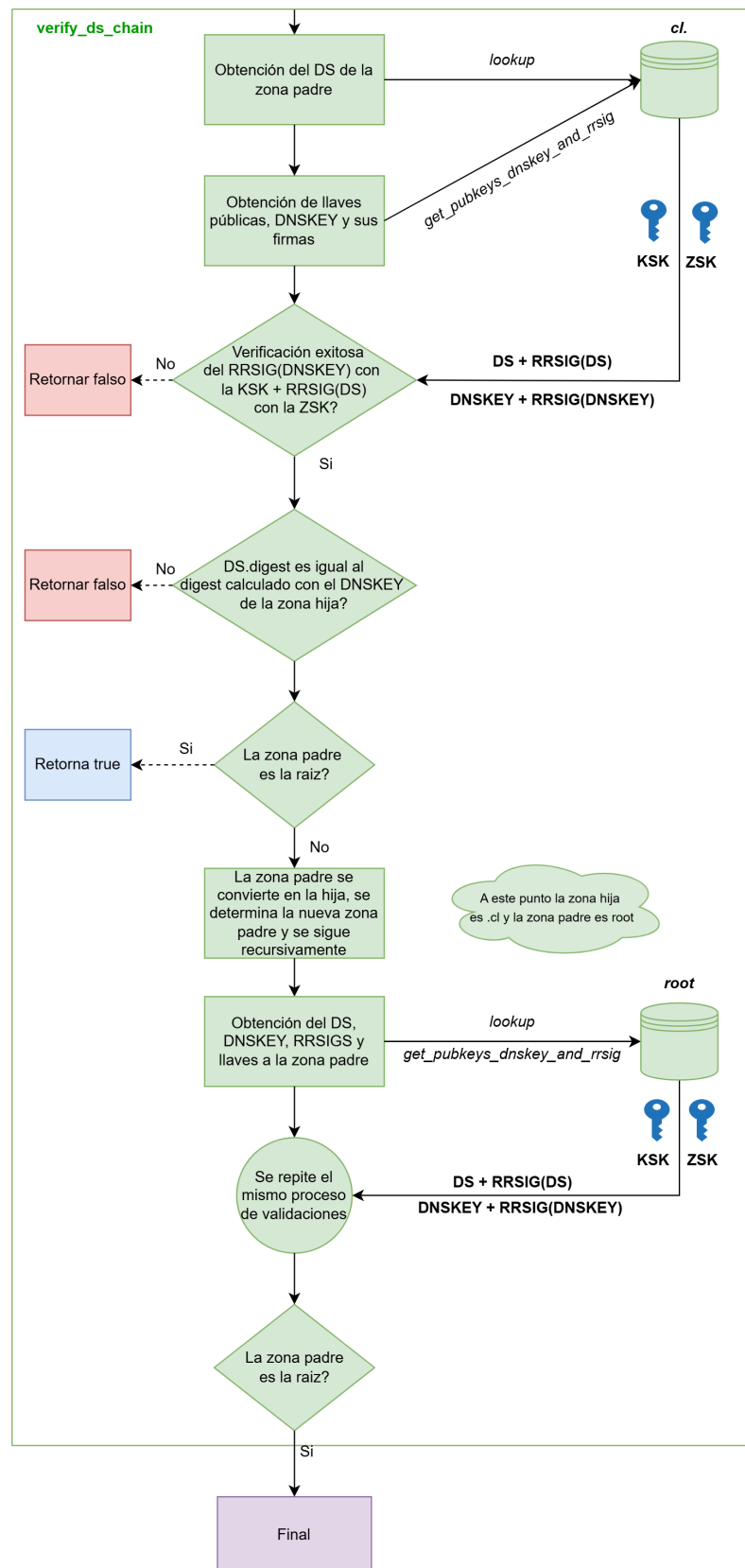
7.2.6. Ejemplo real de validación DNSSEC

Para poder probar que todo lo anterior funciona, se debe crear una consulta DNSSEC válida y usar la función de autenticación para verificar la respuesta. En código sería:

```
1 let dns_query = create_dns_opt_query("niclabs.cl", Rrtype::A);
2 let authentication = dnssec_authentication(&dns_query).await;
3 println!("the authentication is {:?}", authentication);
```

Este proceso retorna verdadero, confirmando que la respuesta DNSSEC es válida y que todas las firmas y la cadena de autenticación han sido verificadas correctamente. Pero además en consola se imprimen todos los pasos intermedios del proceso, mostrando como se obtienen las claves, los **RRsets**, las firmas y los resultados de cada verificación. Mostrar este output completo aquí no es práctico, ya que es demasiado extenso, por lo que a continuación se muestra un diagrama de flujo que ilustra visualmente el proceso de validación.





Como se observa en el diagrama, el proceso de validación DNSSEC es complejo y requiere múltiples pasos y verificaciones para garantizar la autenticidad e integridad de las respuestas

DNS, específicamente requiere de muchos llamados a las distintas base de datos de las zonas involucradas. Aclarar que en el diagrama se pueden ver flechas punteadas, estas indican el camino alternativo en caso de que alguna verificación falle, lo que hace que todo el proceso retorne falso inmediatamente, pero en este ejemplo en concreto todas las verificaciones son exitosas y se llega al final del diagrama retornando verdadero.

También decir que este proceso sería muy similar al que se haría si se tuviera comunicación TCP/UDP entre servidor y resolver. Lo único distinto sería que en vez de obtener las respuestas con llamados a la función `lookup_data`, se haría enviando consultas DNS a través de la red, teniendo que además hacer parsing de las respuestas recibidas. Pero en términos de lógica de validación, sería idéntico.

Así como este proceso retorna verdadero para una consulta A válida, también se probó con todas las siguientes consultas:

Consulta con resultado NXDOMAIN

```
1 let dns_query = create_dns_opt_query("unknown.niclabs.cl", Rrtype  
  ::A);  
2 let authentication = dnssec_authentication(&dns_query).await;
```

Este caso sería el que usa la función `nxdomain_authentication_process` y retorna verdadero, confirmando que la respuesta NXDOMAIN es válida bajo DNSSEC.

Consulta con resultado NO_DATA que no es un delegation point

```
1 let dns_query = create_dns_opt_query("www.example.com", Rrtype::  
  TXT);  
2 let authentication = dnssec_authentication(&dns_query).await;
```

Este caso sería el que usa la función `nodata_authentication_process` y retorna verdadero, confirmando que la respuesta NO_DATA es válida bajo DNSSEC.

Consulta con resultado NO_DATA que es un delegation point

```
1 let dns_query = create_dns_opt_query("b.example.com", Rrtype::A);  
2 let authentication = dnssec_authentication(&dns_query).await;
```

Este caso también usa la función `nodata_authentication_process` pero en este caso se añade el DS asociado a la delegación. Retorna verdadero, confirmando que la respuesta NO_DATA es válida bajo DNSSEC.

Consulta de tipo DNSKEY

```
1     let dns_query = create_dns_opt_query("niclabs.cl", Rrtype::DNSKEY
2     );
    let authentication = dnssec_authentication(&dns_query).await;
```

Este caso activa el modo especial de autenticación para consultas DNSKEY en la función `authentication_process`, usando la KSK para verificar la firma del RRset DNSKEY. Retorna verdadero, confirmando que la respuesta DNSKEY es válida bajo DNSSEC.

Consulta generada por wildcard

```
1     let dns_query = create_dns_opt_query("unknown.example.com",
    Rrtype::A);
2     let authentication = dnssec_authentication(&dns_query).await;
```

En este caso hay que quitar el comentario del registro wildcard en el masterfile de la zona `example.com` para que funcione. Este caso usa la función `wildcard_authentication_process` y retorna verdadero, confirmando que la respuesta generada por el wildcard es válida bajo DNSSEC.

Consulta que utiliza un CNAME

```
1     let dns_query = create_dns_opt_query("blog.example.com", Rrtype::
    A);
2     let authentication = dnssec_authentication(&dns_query).await;
```

Este caso ocupa la función estándar `authentication_process`, pero lo distinto es que contiene dos respuestas de dos tipos distintos: un CNAME y un A. Ambos registros tienen su respectivo RRSIG y ambos son validados correctamente. Retorna verdadero, confirmando que la respuesta con CNAME es válida bajo DNSSEC.

Capítulo 8

Discusiones

Bibliografía

[1] Autor, Título del trabajo, Editorial o Conferencia, Año.

Bibliografía

- [1] Ioannis Karatzas. and Steven E. Shreve. *Brownian Motion and Stochastic Calculus*. Springer, Berlin, 2nd edition, 2000.
- [2] Philip Protter. *Stochastic Integration and Differential Equations*. Springer, 1990.
- [3] Daniel Revuz and Marc Yor. *Continuous martingales and Brownian motion*. Number 293 in Grundlehren der mathematischen Wissenschaften. Springer, Berlin [u.a.], 3. ed edition, 1999.

Apéndice A

Anexo

Quisque facilisis auctor sapien. Pellentesque gravida hendrerit lectus. Mauris rutrum sodales sapien. Fusce hendrerit sem vel lorem. Integer pellentesque massa vel augue. Integer elit tortor, feugiat quis, sagittis et, ornare non, lacus. Vestibulum posuere pellentesque eros. Quisque venenatis ipsum dictum nulla. Aliquam quis quam non metus eleifend interdum. Nam eget sapien ac mauris malesuada adipiscing. Etiam eleifend neque sed quam. Nulla facilisi. Proin a ligula. Sed id dui eu nibh egetas tincidunt. Suspendisse arcu.

Maecenas dui. Aliquam volutpat auctor lorem. Cras placerat est vitae lectus. Curabitur massa lectus, rutrum euismod, dignissim ut, dapibus a, odio. Ut eros erat, vulputate ut, interdum non, porta eu, erat. Cras fermentum, felis in porta congue, velit leo facilisis odio, vitae consectetur lorem quam vitae orci. Sed ultrices, pede eu placerat auctor, ante ligula rutrum tellus, vel posuere nibh lacus nec nibh. Maecenas laoreet dolor at enim. Donec molestie dolor nec metus. Vestibulum libero. Sed quis erat. Sed tristique. Duis pede leo, fermentum quis, consectetur eget, vulputate sit amet, erat.

Donec vitae velit. Suspendisse porta fermentum mauris. Ut vel nunc non mauris pharetra varius. Duis consequat libero quis urna. Maecenas at ante. Vivamus varius, wisi sed egetas tristique, odio wisi luctus nulla, lobortis dictum dolor ligula in lacus. Vivamus aliquam, urna sed interdum porttitor, metus orci interdum odio, sit amet euismod lectus felis et leo. Praesent ac wisi. Nam suscipit vestibulum sem. Praesent eu ipsum vitae pede cursus venenatis. Duis sed odio. Vestibulum eleifend. Nulla ut massa. Proin rutrum mattis sapien. Curabitur dictum gravida ante.

Phasellus placerat vulputate quam. Maecenas at tellus. Pellentesque neque diam, dignissim ac, venenatis vitae, consequat ut, lacus. Nam nibh. Vestibulum fringilla arcu mollis arcu. Sed et turpis. Donec sem tellus, volutpat et, varius eu, commodo sed, lectus. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque enim arcu, suscipit nec, tempus at, imperdiet vel, metus. Morbi volutpat purus at erat. Donec dignissim, sem id semper tempus, nibh massa eleifend turpis, sed pellentesque wisi purus sed libero. Nullam lobortis tortor vel risus. Pellentesque consequat nulla eu tellus. Donec velit. Aliquam fermentum, wisi ac rhoncus iaculis, tellus nunc malesuada orci, quis volutpat dui magna id mi. Nunc vel ante. Duis vitae lacus. Cras nec ipsum.

Morbi nunc. Aliquam consectetur varius nulla. Phasellus eros. Cras dapibus porttitor risus. Maecenas ultrices mi sed diam. Praesent gravida velit at elit vehicula porttitor. Phasellus nisl mi, sagittis ac, pulvinar id, gravida sit amet, erat. Vestibulum est. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur id sem elementum leo rutrum hendrerit. Ut at mi. Donec tincidunt faucibus massa. Sed turpis quam, sollicitudin a, hendrerit eget, pretium ut, nisl. Duis hendrerit ligula. Nunc pulvinar congue urna.

Nunc velit. Nullam elit sapien, eleifend eu, commodo nec, semper sit amet, elit. Nulla lectus risus, condimentum ut, laoreet eget, viverra nec, odio. Proin lobortis. Curabitur dictum arcu vel wisi. Cras id nulla venenatis tortor congue ultrices. Pellentesque eget pede. Sed eleifend sagittis elit. Nam sed tellus sit amet lectus ullamcorper tristique. Mauris enim sem, tristique eu, accumsan at, scelerisque vulputate, neque. Quisque lacus. Donec et ipsum sit amet elit nonummy aliquet. Sed viverra nisl at sem. Nam diam. Mauris ut dolor. Curabitur ornare tortor cursus velit.

Morbi tincidunt posuere arcu. Cras venenatis est vitae dolor. Vivamus scelerisque semper mi. Donec ipsum arcu, consequat scelerisque, viverra id, dictum at, metus. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut pede sem, tempus ut, porttitor bibendum, molestie eu, elit. Suspendisse potenti. Sed id lectus sit amet purus faucibus vehicula. Praesent sed sem non dui pharetra interdum. Nam viverra ultrices magna.

Aenean laoreet aliquam orci. Nunc interdum elementum urna. Quisque erat. Nullam tempor neque. Maecenas velit nibh, scelerisque a, consequat ut, viverra in, enim. Duis magna. Donec odio neque, tristique et, tincidunt eu, rhoncus ac, nunc. Mauris malesuada malesuada elit. Etiam lacus mauris, pretium vel, blandit in, ultricies id, libero. Phasellus bibendum erat ut diam. In congue imperdiet lectus.

Aenean scelerisque. Fusce pretium porttitor lorem. In hac habitasse platea dictumst. Nulla sit amet nisl at sapien egestas pretium. Nunc non tellus. Vivamus aliquet. Nam adipiscing euismod dolor. Aliquam erat volutpat. Nulla ut ipsum. Quisque tincidunt auctor augue. Nunc imperdiet ipsum eget elit. Aliquam quam leo, consectetur non, ornare sit amet, tristique quis, felis. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque interdum quam sit amet mi. Pellentesque mauris dui, dictum a, adipiscing ac, fermentum sit amet, lorem.

Ut quis wisi. Praesent quis massa. Vivamus egestas risus eget lacus. Nunc tincidunt, risus quis bibendum facilisis, lorem purus rutrum neque, nec porta tortor urna quis orci. Aenean aliquet, libero semper volutpat luctus, pede erat lacinia augue, quis rutrum sem ipsum sit amet pede. Vestibulum aliquet, nibh sed iaculis sagittis, odio dolor blandit augue, eget mollis urna tellus id tellus. Aenean aliquet aliquam nunc. Nulla ultricies justo eget orci. Phasellus tristique fermentum leo. Sed massa metus, sagittis ut, semper ut, pharetra vel, erat. Aliquam quam turpis, egestas vel, elementum in, egestas sit amet, lorem. Duis convallis, wisi sit amet mollis molestie, libero mauris porta dui, vitae aliquam arcu turpis ac sem. Aliquam aliquet dapibus metus.

Vivamus commodo eros eleifend dui. Vestibulum in leo eu erat tristique mattis. Cras at elit. Cras pellentesque. Nullam id lacus sit amet libero aliquet hendrerit. Proin placerat, mi non elementum laoreet, eros elit tincidunt magna, a rhoncus sem arcu id odio. Nulla eget leo a

leo egestas facilisis. Curabitur quis velit. Phasellus aliquam, tortor nec ornare rhoncus, purus urna posuere velit, et commodo risus tellus quis tellus. Vivamus leo turpis, tempus sit amet, tristique vitae, laoreet quis, odio. Proin scelerisque bibendum ipsum. Etiam nisl. Praesent vel dolor. Pellentesque vel magna. Curabitur urna. Vivamus congue urna in velit. Etiam ullamcorper elementum dui. Praesent non urna. Sed placerat quam non mi. Pellentesque diam magna, ultricies eget, ultrices placerat, adipiscing rutrum, sem.